System Design Mastery: Complete Placement Preparation Guide

A Premium Technical Reference for Software Engineering Interviews

---

> **How to use this guide:** Read Chapters 1–8 for foundational concepts. Study Chapter 9 for HLD deep dives. Use Chapters 10–11 for LLD. Review Chapter 12 for Q&A. Skim Chapter 14 the morning of your interview.

Table of Contents

#	Chapter	Key Topics
1	Chapter 1: System Design Fundamentals & Interview Framework	HLD vs LLD, CAP Theorem, Estimations, Latency Numbers
2	Chapter 2: Scalability & Reliability Concepts	Load Balancing, Caching, Sharding, Rate Limiting, Circuit Breaker
3	Chapter 3: Database Design & Data Storage	SQL, NoSQL, Indexing, Partitioning, Blob Storage
4	Chapter 4: Microservices & System Architecture	Monolith vs Microservices, API Gateway, Service Mesh, CQRS
5	Chapter 5: Message Queues & Event-Driven Architecture	Kafka, RabbitMQ, Delivery Semantics, DLQ
6	Chapter 6: Security & Authentication	JWT, OAuth 2.0, TLS, Encryption, CORS
7	Chapter 7: Monitoring, Logging & Observability	Metrics, Traces, Logs, SLI/SLO/SLA
8	Chapter 8: Cloud & DevOps Concepts	Kubernetes, Serverless, CI/CD, IaC
9	Chapter 9: HLD Deep Dives — Designing 20 Systems	URL Shortener, Chat, YouTube, Uber, Payment + 15 more
10	Chapter 10: Low-Level Design (LLD) Fundamentals	SOLID, Design Patterns (23 GoF), OOD Interview Problems

#	Chapter	Key Topics
11	Chapter 11: Advanced LLD	Concurrency, DDD, Clean Architecture, Refactoring
12	Chapter 12: Top 40 Interview Q&A	40 Most-Asked System Design Questions with Detailed Answers
13	Chapter 13: Practice Platforms & Resources	Study platforms, YouTube videos, Books
14	Chapter 14: System Design Glossary A–Z	80+ terms defined

Chapter 1: System Design Fundamentals & Interview Framework {#chapter-1}

1.1 What is System Design?

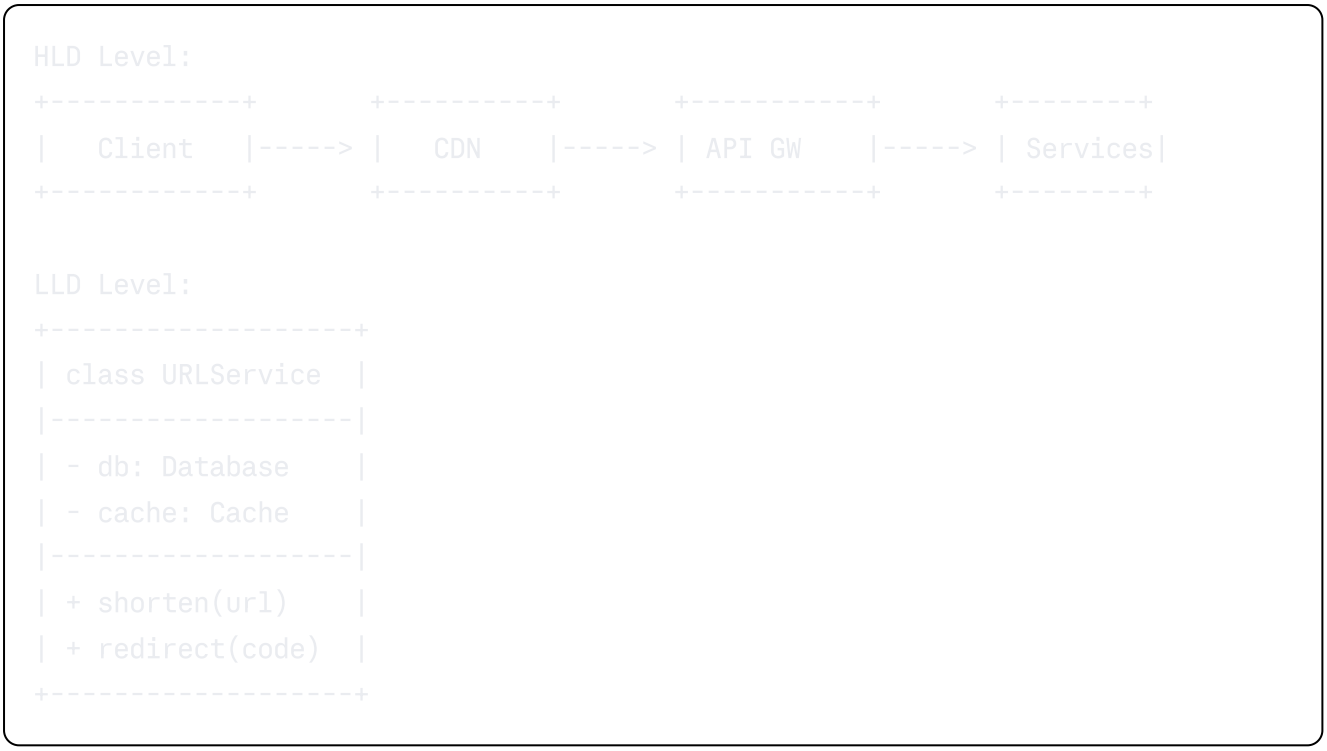
System Design is the process of defining the architecture, components, modules, interfaces, and data flow of a software system to satisfy specified requirements. Unlike algorithmic problems — where a single optimal solution usually exists — system design problems are inherently open-ended, involving trade-offs between competing goals such as consistency vs availability, latency vs throughput, and cost vs reliability.

In the context of software engineering interviews, system design evaluates your ability to think at scale. Interviewers are not looking for a "perfect" answer; they want to see structured thinking, awareness of constraints, and the ability to reason through trade-offs.

1.1.1 High-Level Design (HLD) vs Low-Level Design (LLD)

High-Level Design (HLD) addresses the macro-architecture of a system. It deals with how major components interact, what databases and caches to use, how traffic flows through the system, and how the system achieves non-functional requirements like availability and scalability. HLD operates at the level of services, APIs, and infrastructure.

Low-Level Design (LLD) zooms into the micro-architecture — class structures, method signatures, design patterns, database schemas, and object-oriented relationships. LLD is about how you would actually code the solution, including which design patterns apply and how classes collaborate.



In placement interviews, most companies ask HLD for 45–60 minute rounds and LLD for separate 45-minute coding rounds. Senior engineers are expected to navigate both fluently.

1.2 The System Design Interview Process

A structured interview framework prevents rambling and demonstrates seniority. Follow this 7-step process every time:

Step 1: Requirements Clarification (5 minutes)

Before writing a single component, ask clarifying questions. Interviewers intentionally leave requirements vague. Questions to ask:

- "What is the scale? How many daily active users?"
- "Is this read-heavy or write-heavy?"
- "Do we need strong consistency or is eventual consistency acceptable?"
- "What are the latency requirements (P99 < 200ms)?"
- "Should we focus on the core feature or include analytics, notifications, etc.?"
- "What is the expected storage growth per year?"

Step 2: Back-of-the-Envelope Estimation (5 minutes)

Quantify the system's scale. This drives every architectural decision. Calculate: QPS, storage, bandwidth, memory needed. (Detailed in Section 1.4.)

Step 3: API Design (5 minutes)

Define the public interface. REST endpoints, request/response payloads, authentication headers. This forces you to think about the system's contract before its internals.

Step 4: Data Model (5 minutes)

Choose your database type (SQL vs NoSQL), define the schema (tables, fields, indexes), and justify the choice based on your access patterns.

Step 5: High-Level Design (15 minutes)

Draw the system at the service level. Show: client → CDN → load balancer → app servers → caches → databases → message queues. Walk the interviewer through request flow.

Step 6: Deep Dive (10 minutes)

The interviewer will pick 1–2 components to explore deeply. Common deep-dives: "How does your database handle 10x traffic?" or "Walk me through the message delivery guarantee in your queue."

Step 7: Trade-offs & Alternatives (5 minutes)

Proactively discuss what you would do differently, what you sacrificed, and what you would change if the scale doubled or halved.

> **Interview Tip:** Never jump to drawing boxes. Spend at least 5 minutes on requirements. Interviewers frequently give hints through their follow-up questions — listen carefully.

1.3 Functional vs Non-Functional Requirements

Every system design starts by separating what the system **does** (functional) from how well it **performs** (non-functional).

Functional Requirements

These are the features — the "what." For a URL shortener:

- Users can shorten a URL
- Users can be redirected via the short URL
- Users can set an expiry on short URLs
- Users can see click analytics

Non-Functional Requirements (NFRs)

These define the quality attributes — the "how well":

NFR	Definition	Example Target
Availability	% of time the system is operational	99.99% (52.6 min downtime/year)
Scalability	Ability to handle growing load	Handle 10x traffic with same latency
Reliability	System performs correctly over time	No data loss, durable writes
Performance	Speed of operations	P99 read latency < 50ms
Consistency	All nodes see the same data	Reads always return latest write
Durability	Data persists despite failures	Data survives node crashes

Availability is typically expressed in "nines":

```
99%    = 87.6 hours/year downtime
99.9%  = 8.76 hours/year downtime
99.99% = 52.6 minutes/year downtime
99.999%= 5.26 minutes/year downtime ("five nines")
```

Scalability has two dimensions: *horizontal* (more machines) and *vertical* (bigger machines). True scalability means performance degrades gracefully — not catastrophically — under increased load.

Reliability differs from availability. A system can be available (responding) but unreliable (returning wrong data). Reliability encompasses both correctness and fault-tolerance.

1.4 Back-of-the-Envelope Calculations

Estimation is a critical interview skill. The goal is not precision — it is an order-of-magnitude understanding that shapes architectural decisions. Always state your assumptions explicitly.

Key Constants to Memorize

```
1 KB  = 10^3  bytes   = 1,000 bytes
1 MB  = 10^6  bytes   = 1,000,000 bytes
1 GB  = 10^9  bytes   = 1,000,000,000 bytes
1 TB  = 10^12 bytes
1 PB  = 10^15 bytes
```

```
Seconds in a day:   86,400 ≈ 10^5
Seconds in a month: 2,592,000 ≈ 2.5 × 10^6
Seconds in a year:  31,536,000 ≈ 3 × 10^7
```

QPS / RPS Estimation

Example: Twitter-scale News Feed

- Assumption: 300 million MAU, 50% DAU = 150 million DAU
- Average user reads feed 10 times/day → 1.5 billion read requests/day
- QPS (reads) = 1,500,000,000 / 86,400 ≈ **17,400 QPS** (peak: ~35,000 QPS)
- Tweets per day: 150M users × 0.1% post = 150,000 tweets/day ≈ **1.74 write QPS**

This immediately tells us the system is *heavily read-dominated* (~10,000:1 read:write ratio).

Storage Estimation

Example: URL Shortener

- 100 million new URLs/day
- Average URL length: 100 bytes
- Short URL + metadata: 500 bytes/record
- Daily storage: 100,000,000 × 500 = 50 GB/day
- Yearly storage: 50 GB × 365 ≈ **18 TB/year**
- 5-year storage (with 3× replication): 18 TB × 5 × 3 = **270 TB**

Bandwidth Estimation

Example: YouTube

- Uploads: 500 hours of video/minute
- Average video: 1 GB/hour → 500 GB/minute upload bandwidth = **8.3 GB/s**
- Views: 1 billion views/day, average 5 min of video at 4 Mbps
- Download bandwidth: 1B × 5 min × 4 Mbps / 86,400 = **23 Gbps**

Memory/Cache Estimation

Example: Caching hot URLs

- 20% of URLs generate 80% of traffic (Pareto principle)
- If 1 million active URLs, cache 200,000
- Each URL entry: 1 KB
- Cache size needed: 200,000 × 1 KB = **200 MB** (fits easily in a single Redis node)

Number of Servers

```
Servers needed = Peak QPS / Requests handled per server

Assumptions:
- Web server (Node.js/Go): 10,000 RPS
- Application server (Java): 1,000 RPS
- Database server: 1,000 QPS

For 35,000 QPS peak: 35,000 / 10,000 = 4 web servers (add 2× for redundancy =
8)
```

1.5 Latency Numbers Every Programmer Should Know

These numbers, originally from Jeff Dean at Google, are essential for making trade-off decisions. Memorize the orders of magnitude, not exact values.

Operation	Approximate Latency	Notes
L1 cache reference	0.5 ns	CPU on-chip cache
L2 cache reference	7 ns	~14× slower than L1
Mutex lock/unlock	25 ns	OS primitive
Main memory (RAM) reference	100 ns	~200× slower than L1
Compress 1 KB with Snappy	3,000 ns (3 μs)	
Read 1 MB sequentially from RAM	250,000 ns (250 μs)	
Read 4 KB randomly from SSD	150,000 ns (150 μs)	
Read 1 MB sequentially from SSD	1,000,000 ns (1 ms)	~4× slower than RAM
Round trip within same datacenter	500,000 ns (0.5 ms)	
Read 1 MB sequentially from disk (HDD)	20,000,000 ns (20 ms)	~80× slower than SSD
Send packet CA→Netherlands→CA	150,000,000 ns (150 ms)	Cross-continental
DNS lookup	20,000,000 ns (20 ms)	Can be cached

Key takeaways:

- RAM is ~200× faster than SSD; use memory aggressively for hot data
- Network calls within a datacenter (~0.5ms) are cheap; cross-continental (~150ms) are expensive
- Disk I/O (HDD) is the slowest — avoid at all costs in the hot path
- Database queries without indexes can degrade to HDD-speed

1.6 The CAP Theorem

The CAP Theorem (Brewer's Theorem, 2000) states that a distributed data store can guarantee at most **two** of the following three properties simultaneously:

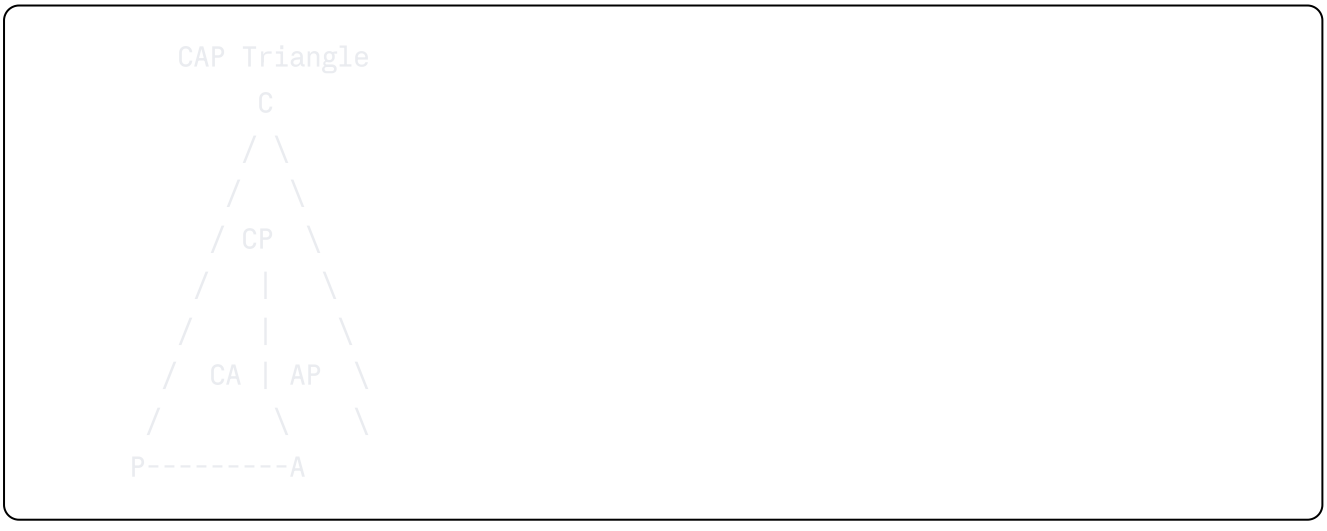
Consistency (C): Every read receives the most recent write or an error. All nodes in the cluster see the same data at the same time.

Availability (A): Every request receives a response (not an error), though it may not contain the most recent write. The system is always operational.

Partition Tolerance (P): The system continues to function even when network partitions occur (messages between nodes are lost or delayed).

Why Only 2 of 3?

In any real distributed system, network partitions *will* occur — this is unavoidable (hardware fails, cables break, switches crash). Therefore, **Partition Tolerance is mandatory**. The real choice is between **Consistency and Availability** during a partition:



When a partition occurs:

- **CP system** (Consistency + Partition Tolerance): Returns an error or waits until data is consistent. Sacrifices availability. *Examples: HBase, Zookeeper, MongoDB (in default config), Redis (primary-replica)*

- **AP system** (Availability + Partition Tolerance): Returns possibly stale data but always responds. Sacrifices consistency. *Examples: Cassandra, CouchDB, DynamoDB (default), DNS*
- **CA system** (Consistency + Availability): Only possible without partitions — i.e., single-node or co-located systems. *Examples: PostgreSQL (single-node), MySQL (single-node)*

Real-World System Categorization

System	CAP Category	Reasoning
Zookeeper	CP	Refuses reads during leader election
Apache Cassandra	AP	Tunable consistency, defaults to eventual
MongoDB	CP (default)	Primary election can make system unavailable
Amazon DynamoDB	AP (default)	Eventually consistent reads by default
Apache HBase	CP	Strong consistency, master failure = outage
MySQL (replicated)	CA→CP	Replication lag = inconsistency during partition
Redis Cluster	CP	Majority quorum required for writes
Couchbase	AP	Eventual consistency, always available

> **Interview Tip:** The CAP theorem is often misunderstood. C in CAP means *linearizability* (the strongest form of consistency), not just "no dirty reads." Many systems offer intermediate consistency levels. When asked "is X CP or AP?", discuss the *default* behavior and note that many systems have tunable consistency.

1.7 PACELC Theorem

The **PACELC Theorem** (Daniel Abadi, 2012) extends CAP by asking: what happens *even when there is no partition*? The theorem states:

- **If there is a Partition (P):** choose between **Availability (A)** and **Consistency (C)** — same as CAP
- **Else (E), when the system is running normally:** choose between **Latency (L)** and **Consistency (C)**

```
PACELC: P → A/C,  E → L/C

Examples:
- Cassandra: PA/EL (sacrifices C for A during partition; sacrifices C for L
normally)
- HBase:     PC/EC (consistent under partition; pays latency for consistency)
- DynamoDB:  PA/EL (available, eventually consistent, low latency)
- MySQL:     PC/EC (single node: no partition, low latency vs consistency
trade-off)
```

PACELC is more nuanced than CAP because latency is a real cost even in the absence of failures. Most cloud databases (Cassandra, DynamoDB) are PA/EL — they sacrifice consistency to achieve both availability during failures and low latency during normal operation.

1.8 The Twelve-Factor App Methodology

The **Twelve-Factor App** (Heroku, 2012) is a methodology for building software-as-a-service applications that are scalable, maintainable, and portable. Relevant in system design interviews when discussing deployment, configuration, and cloud-native design.

Factor	Principle
1. Codebase	One codebase tracked in version control; many deploys
2. Dependencies	Explicitly declare and isolate dependencies
3. Config	Store config in environment variables, not code
4. Backing Services	Treat databases, queues, etc. as attached resources
5. Build, Release, Run	Strictly separate build and run stages
6. Processes	Execute app as stateless processes (no sticky sessions)
7. Port Binding	Export services via port binding
8. Concurrency	Scale out via process model
9. Disposability	Fast startup, graceful shutdown
10. Dev/Prod Parity	Keep development and production as similar as possible
11. Logs	Treat logs as event streams
12. Admin Processes	Run admin tasks as one-off processes

Chapter 1 — Quick Revision Box

> - **HLD** = macro-architecture (services, APIs, databases); **LLD** = micro-architecture (classes, patterns)
> - Interview framework: Clarify → Estimate → API → Data Model → HLD → Deep Dive → Trade-offs
> - **Availability ≠ Reliability**: a system can be up (available) but returning stale data (unreliable)
> - **CAP**: distributed systems must choose between C and A during a network partition; P is not optional
> - **PACELC**: extends CAP — even without partitions, there's a Latency vs Consistency trade-off
> - Memorize latency order: L1 cache (0.5ns) → RAM (100ns) → SSD (150µs) → HDD (20ms) → Network (0.5ms–150ms)
> - Back-of-envelope math: state assumptions, use round numbers, derive QPS → storage → servers → bandwidth

Chapter 2: Scalability & Reliability Concepts {#chapter-2}

2.1 Vertical Scaling (Scale Up)

Vertical scaling means adding more resources — CPU, RAM, faster disks — to an existing machine. It is the simplest form of scaling because it requires no changes to application code or architecture.

When to use: Early-stage applications with predictable load, stateful systems (like single-node databases) that are hard to distribute, and when simplicity is more valuable than theoretical throughput limits.

Limits: Every machine has a physical ceiling. The largest available server (as of 2024) offers ~448 cores and ~24 TB RAM. Beyond that, vertical scaling is impossible. Additionally, vertical scaling means a single point of failure — if the machine crashes, the entire system goes down.

```
Vertical Scaling:
[Small Server]  →  [Medium Server]  →  [Large Server]
  4 CPU / 8 GB       16 CPU / 64 GB      96 CPU / 512 GB
  (simple, SPOF)
```

2.2 Horizontal Scaling (Scale Out)

Horizontal scaling means adding more machines to distribute the load. This is the foundation of modern large-scale systems. It requires that application servers be **stateless** — no session state stored locally on the server — so that any request can be routed to any server.

Advantages: No theoretical ceiling, improved fault tolerance (one node failing doesn't bring down the system), cost-effective (use commodity hardware).

Challenges: Requires a load balancer, stateless application design, distributed coordination, and more complex operations.

```
Horizontal Scaling:
                    +---> App Server 1 ---> DB
Client --> LB  ----|---> App Server 2 ---> DB
                    +---> App Server 3 ---> DB
```

A **load balancer** is an intermediary that distributes incoming traffic across multiple servers to ensure no single server becomes a bottleneck. It also provides health checking — removing unhealthy servers from the pool automatically.

Layer 4 vs Layer 7 Load Balancing

Dimension	Layer 4 (Transport)	Layer 7 (Application)
OSI Layer	Transport (TCP/UDP)	Application (HTTP/HTTPS)
Routing basis	IP address + port	HTTP headers, URL, cookies, content
TLS Termination	No (passes through)	Yes (decrypts at LB)
Speed	Faster (less processing)	Slower (full packet inspection)
Examples	AWS NLB, HAProxy (TCP mode)	AWS ALB, Nginx, HAProxy (HTTP mode)
Use case	Raw TCP apps, gaming, streaming	Web apps, APIs, microservices

Load Balancing Algorithms

Round Robin: Requests are distributed to servers in sequential order. Simple and effective when servers have equal capacity.

```
Request 1 → Server A
Request 2 → Server B
Request 3 → Server C
Request 4 → Server A  (cycle repeats)
```

Weighted Round Robin: Like Round Robin, but servers with higher capacity receive proportionally more requests. If Server A has 4× the capacity of Server B, set weight A=4, B=1.

Least Connections: Each new request goes to the server with the fewest active connections. Best for requests with varying processing times (long-lived connections).

Least Response Time: Routes to the server with the lowest average response time and fewest connections. More sophisticated than least connections.

IP Hash: The client's IP address is hashed to determine which server handles the request. This ensures the same client always reaches the same server — useful for stateful applications. Note: fails if a server is added or removed (changes hash modulo).

Health Checks

Load balancers continuously probe backend servers with health check requests (e.g., HTTP GET `/health`). If a server fails N consecutive checks, it is removed from the pool. When it recovers, it is re-added.

Sticky Sessions (Session Affinity)

A configuration where a load balancer routes all requests from a given client to the same backend server, typically via a cookie. Needed for stateful applications. *Downside:* defeats the purpose of horizontal scaling and causes uneven load if sessions vary in intensity.

Hardware vs Software Load Balancers

Type	Examples	Pros	Cons
Hardware	F5 BIG-IP, Citrix ADC	Very high performance, dedicated hardware	Expensive ($20K–$100K+), inflexible
Software	Nginx, HAProxy, Envoy	Flexible, cheap, cloud-friendly	Needs tuning, software overhead
Cloud-managed	AWS ALB/NLB, GCP Cloud LB	Fully managed, auto-scaling	Vendor lock-in

2.4 Caching

Caching is storing copies of frequently accessed data in fast storage (typically in-memory) to reduce latency and database load. A cache **hit** occurs when the requested data is found in cache; a **cache miss** occurs when it is not, requiring a slower backend fetch.

Cache effectiveness is measured by the **hit rate:** `hit rate = cache hits / total requests`. A 95% hit rate means 95% of requests are served from cache.

Cache Placement

```
Request Flow with Cache Layers:
```

```
Browser Cache → CDN Cache → Reverse Proxy Cache → App Cache → DB Cache →
Database
(fastest)
(slowest)
```

- **Client-side cache:** Browser stores static assets (JS, CSS, images). Controlled via `Cache-Control` and `ETag` headers.
- **CDN cache:** Geographically distributed edge nodes cache static and semi-static content close to users.
- **Reverse proxy cache (e.g., Nginx, Varnish):** Caches entire HTTP responses at the infrastructure level.
- **Application-level cache:** In-process memory (e.g., Caffeine in Java, functools.lru_cache in Python) or external distributed cache (Redis, Memcached).
- **Database cache:** Query result cache (deprecated in MySQL 8.0), buffer pool (InnoDB), row cache.

Cache Replacement Policies

Policy	How It Works	Best For
LRU (Least Recently Used)	Evicts the item not accessed for the longest time	General-purpose; works well when recent items are likely to be reused
LFU (Least Frequently Used)	Evicts the item accessed the fewest times	When popularity doesn't change — once-popular items aren't kept forever
FIFO (First In, First Out)	Evicts the oldest-inserted item	Simple; not great for hot items
Random	Evicts a random item	Surprisingly effective; low overhead

Cache Invalidation Strategies

Write-through: Data is written to cache and database simultaneously. Cache is always consistent with DB. *Downside:* every write hits the DB, so write performance is limited by DB throughput.

```
Write-through:
Client → Write to Cache → Write to DB → Acknowledge
(Reads always hit cache, cache = DB)
```

Write-around: Data is written directly to DB, bypassing cache. Cache is only populated on reads. *Problem:* First read after write is always a cache miss.

Write-back (Write-behind): Data is written to cache first, then asynchronously written to DB. Very fast writes. *Risk:* Data loss if cache crashes before write-back completes.

```
Write-back:
Client → Write to Cache → Acknowledge (fast)
Cache → [asynchronously] → Write to DB (background)
```

Cache-aside (Lazy Loading): The application manages the cache explicitly. On a cache miss, the application fetches from DB, writes to cache, and returns data. Most common pattern in practice.

```
Cache-aside read:
Client → Check Cache
  HIT:  Cache → Return data
  MISS: DB → Fetch data → Write to Cache → Return data
```

Read-through: The cache itself is responsible for fetching from DB on misses. Application always talks to the cache. Simpler application code; used by frameworks like Ehcache.

Cache Stampede (Thundering Herd)

When a popular cache entry expires, hundreds or thousands of simultaneous requests miss the cache, all hitting the database at once. Solutions:

- **Probabilistic Early Expiration (XFetch):** Refresh cache slightly before expiry with some probability
- **Mutex/Lock:** Only one thread fetches from DB; others wait
- **Background refresh:** Refresh cache asynchronously while serving stale data temporarily

Distributed Cache: Redis vs Memcached

Feature	Redis	Memcached
Data structures	Strings, Lists, Sets, Sorted Sets, Hashes, Streams, HyperLogLog	Strings only
Persistence	RDB snapshots + AOF log	None (pure in-memory)
Replication	Yes (primary-replica + sentinel)	No native replication

Feature	Redis	Memcached
Clustering	Redis Cluster (sharding + replication)	Client-side sharding only
Pub/Sub	Yes	No
Lua scripting	Yes	No
Performance	Slightly slower (more features)	Marginally faster for simple get/set
Use case	Sessions, leaderboards, queues, pub/sub, rate limiting	Simple distributed object cache

Recommendation: Default to Redis. Only consider Memcached if you need maximum throughput for purely string key-value caching with no persistence requirements.

2.5 Content Delivery Network (CDN)

A **CDN** is a geographically distributed network of servers (edge nodes) that cache and serve content from locations close to end users, reducing latency and offloading traffic from the origin server.

Pull CDN: Content is cached at the edge *on first request*. The CDN fetches from origin on a cache miss. Simple to set up; the CDN "pulls" content when needed. Example: CloudFront (default), Fastly.

Push CDN: You push content to the CDN proactively. Content is available at edge nodes before any user requests it. Better for large files (videos, software downloads) where you want zero-latency first access.

Cache Invalidation in CDNs: CDNs typically cache based on TTL (Time-to-Live). To invalidate before TTL expiry: use URL versioning (`style.v2.css`), query strings (`?v=123`), or explicit CDN API calls to purge specific paths.

2.6 Database Scaling

Read Replicas (Master-Slave Replication)

Write operations go to the **primary (master)** node, which replicates data to one or more **replica (slave)** nodes. Read operations are distributed across replicas, scaling read capacity horizontally.

Synchronous replication: Primary waits for replica to acknowledge before returning success. Zero replication lag but higher write latency. *Asynchronous replication:* Primary acknowledges immediately; replica syncs in background. Lower write latency but risk of data loss and replication lag.

Replication lag is the delay between a write on primary and its visibility on replicas. During high write loads, lag can reach seconds or minutes, causing users to read stale data.

Database Sharding

Sharding is horizontal partitioning of data across multiple database servers, where each shard holds a subset of the total data. Unlike replication (where every node has all data), sharding distributes different data to different nodes.

Sharding strategies:

Range-based sharding: Shard by a value range (e.g., user IDs 1–1M on Shard 1, 1M–2M on Shard 2). Simple to understand but can lead to hotspots if data isn't uniformly distributed (e.g., new user IDs always go to the last shard).

Hash-based sharding: Apply a hash function to the shard key (e.g., `hash(user_id) % num_shards`). Uniform distribution but makes range queries across shards impossible.

Geographic sharding: Route data based on user location (e.g., European users on EU shard). Reduces latency and enables data residency compliance (GDPR).

Directory-based sharding: A lookup service (directory) maps shard keys to specific shards. Most flexible — you can migrate data between shards without changing application code. *Downside:* The directory itself becomes a potential single point of failure.

Shard key selection is critical. A bad shard key leads to:

- **Hotspots:** One shard receives disproportionately more traffic (e.g., sharding by celebrity user ID in Twitter)
- **Cross-shard joins:** Queries that span multiple shards require expensive scatter-gather operations
- **Resharding complexity:** If you outgrow a shard, redistributing data is painful

Resharding: When a shard grows too large, it must be split. Consistent hashing minimizes data movement during resharding (see Chapter 9 — Consistent Hashing).

Federation (Functional Partitioning)

Instead of sharding a single large table, split the database by function or domain: a Users DB, a Products DB, a Orders DB. Each database is independently scalable and has a smaller schema. Cross-domain joins are impossible and must be handled at the application layer.

Denormalization

Denormalization deliberately introduces redundancy into the schema to improve read performance. Instead of joining 5 tables for a common query, pre-compute and store the joined result.

Example: Instead of joining `users`, `orders`, and `products` to display an order history, maintain a denormalized `order_history` table with all fields pre-joined.

Trade-off: Faster reads, but writes are slower (must update multiple places), and data consistency is harder to maintain.

2.7 Rate Limiting

Rate limiting controls how many requests a client can make in a given time window. It protects services from abuse, prevents DoS attacks, and enforces fair resource usage.

Algorithms

Fixed Window Counter: Divide time into fixed windows (e.g., 1-minute windows). Count requests per window per client. Simple but has a boundary problem — a client can send 2× the limit by exploiting the window transition.

```
Window: [0s - 60s] Max: 100 requests
Client sends 100 requests at t=59s and 100 at t=61s → 200 requests in 2 seconds (exploit!)
```

Sliding Window Log: Store a timestamp for each request in a sorted set. To check the limit, count requests within the last [window size]. Accurate but memory-intensive (stores all timestamps).

Sliding Window Counter: Hybrid approach. Uses the current window count and a weighted fraction of the previous window count.

```
Estimate = current_window_count + prev_window_count × (1 - elapsed_fraction)
```

Approximate but memory-efficient and accurate enough for most uses.

Token Bucket: A bucket holds tokens (up to a maximum). Tokens are added at a fixed rate. Each request consumes one token. If the bucket is empty, the request is rejected. Allows bursting (use accumulated tokens for short spikes).

```
Token Bucket:
[fill at 10 tokens/sec, max capacity 100]
Burst: use 100 tokens at once → allowed
Then: must wait for refill before more requests
```

Leaky Bucket: Requests enter a queue (bucket) and are processed at a fixed rate. Excess requests overflow (are dropped). Smooths out bursts — good for maintaining a steady output rate (e.g., sending SMS).

```
Leaky Bucket:
Requests → [Queue/Bucket] → [Process at fixed rate, e.g., 10 req/sec] → Out
Overflow → Drop/reject
```

Comparison Table

Algorithm	Burst Handling	Memory	Accuracy	Best Use Case
Fixed Window	Yes (boundary exploit)	Low	Low	Simple, non-critical
Sliding Window Log	Exact	High	Exact	Low traffic, high precision
Sliding Window Counter	Approximate burst	Low	High	Most production systems
Token Bucket	Yes (controlled)	Low	High	API gateways, user-facing APIs
Leaky Bucket	No (smoothed)	Medium	High	Output rate control (SMS, emails)

Distributed Rate Limiting: In a cluster of API servers, rate limiting must be centralized. Use Redis with atomic operations (`INCR`, `EXPIRE`) or Lua scripts to implement thread-safe counters shared across servers.

2.8 Idempotency

Idempotency means that making the same request multiple times produces the same result as making it once. This is critical in distributed systems where retries are common (due to timeouts and network failures).

Example: `DELETE /users/123` is idempotent — deleting a non-existent user still leaves the system in the same state. `POST /orders` is NOT idempotent — each call creates a new order.

Idempotency keys: For non-idempotent operations (payments, order creation), generate a unique key (UUID) client-side and send it with the request. The server stores the result against this key. Retried requests with the same key return the cached result without re-executing.

```
Client → POST /charge { idempotency_key: "abc123", amount: 500 }
Server → Process charge (1st time) → Store result for "abc123" → Return 200
Client → POST /charge { idempotency_key: "abc123", amount: 500 } (retry)
Server → "abc123" already processed → Return cached 200 (no duplicate charge)
```

2.9 Circuit Breaker Pattern

The **Circuit Breaker** pattern prevents a system from repeatedly calling a failing service, giving it time to recover. Named after electrical circuit breakers.

Implementation: Track the number (or rate) of failures in a sliding window. When the failure rate exceeds a threshold (e.g., 50% of requests fail in the last 60 seconds), open the circuit. After a timeout (e.g., 30 seconds), transition to Half-Open and test with limited traffic.

Libraries: Resilience4j (Java), Hystrix (Netflix, deprecated), Polly (.NET), `pybreaker` (Python).

2.10 Bulkhead Pattern

Bulkhead (named after ship compartments that prevent flooding) isolates different consumers or functions into separate pools of resources. If one pool is exhausted, other pools remain unaffected.

Example: Separate thread pools for: (a) user-facing API requests, (b) batch processing jobs, (c) third-party service calls. A slow third-party service only exhausts its pool — the user-facing threads remain available.

2.11 Retry with Exponential Backoff and Jitter

Exponential backoff spaces out retries with increasing delays to avoid overloading a

recovering service.

```
Retry delays: 1s → 2s → 4s → 8s → 16s → give up (max 5 retries)
Formula: delay = base_delay × 2^retry_count
```

Jitter adds randomness to the delay to prevent **retry storms** — where all clients retry at the same instant after a service recovery.

```
With jitter: delay = random_between(0, base_delay × 2^retry_count)
```

This spreads retry traffic over time, preventing the service from being immediately overwhelmed upon recovery.

Chapter 2 — Quick Revision Box

> - **Vertical scaling** has physical limits and a SPOF; **horizontal scaling** is theoretically infinite but requires stateless services
> - **Load balancer algorithms:** Round Robin (equal servers), Least Connections (variable request times), IP Hash (session affinity), Weighted (heterogeneous servers)
> - **Cache-aside** is the most common pattern; write-back has best write performance but risks data loss
> - **Redis beats Memcached** for almost every use case due to richer data structures, persistence, and built-in clustering
> - **Sharding strategies:** Range (simple, hotspot risk) → Hash (uniform, no range queries) → Geographic (compliance) → Directory (flexible, SPOF risk)
> - **Token bucket** allows bursting; **leaky bucket** enforces steady rate; **sliding window counter** is the best general-purpose rate limiter
> - **Circuit breaker states:** Closed (normal) → Open (fail-fast) → Half-Open (test recovery)

Chapter 3: Database Design & Data Storage {#chapter-3}

3.1 Relational Databases (SQL)

Relational databases organize data into tables (relations) with predefined schemas. They are the cornerstone of transactional applications and provide strong consistency

guarantees through ACID properties.

ACID Properties

Atomicity: A transaction is all-or-nothing. Either all operations in a transaction succeed or none do. If a bank transfer debits Account A but the system crashes before crediting Account B, the entire transaction is rolled back.

Consistency: A transaction brings the database from one valid state to another valid state, respecting all integrity constraints (foreign keys, unique constraints, check constraints).

Isolation: Concurrent transactions execute as if they were serial. Controlled by isolation levels:

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	✓ Possible	✓ Possible	✓ Possible
Read Committed	✗ Prevented	✓ Possible	✓ Possible
Repeatable Read	✗ Prevented	✗ Prevented	✓ Possible
Serializable	✗ Prevented	✗ Prevented	✗ Prevented

Durability: Once a transaction commits, it persists even if the system crashes immediately after. Achieved via write-ahead logging (WAL) — changes are written to a log before being applied to data files.

Database Indexing

An **index** is a data structure that speeds up read queries at the cost of additional storage and slower writes (the index must be updated on every write).

B+ Tree Index: The default index type in most relational databases. A balanced tree where all data is in the leaf nodes, connected by a linked list for efficient range scans.

Types of indexes:

- **Primary index:** Built on the primary key. Data rows are physically ordered by primary key (clustered index in MySQL/InnoDB).
- **Secondary index:** Built on non-key columns. Points to primary key values (not physical row locations), so a secondary index lookup involves two tree traversals.
- **Composite index:** Index on multiple columns. Order matters — `INDEX(a, b, c)` can serve queries on `(a)`, `(a, b)`, and `(a, b, c)` but NOT `(b)` or `(c)` alone.
- **Covering index:** An index that contains all columns needed by a query, allowing the query to be satisfied entirely from the index without touching the data rows.

Index selectivity: High-selectivity indexes (like user_id) are effective; low-selectivity indexes (like boolean `is_active`) offer little benefit and can slow down writes.

3.2 NoSQL Databases

NoSQL ("Not Only SQL") databases sacrifice some ACID guarantees for horizontal scalability, flexible schemas, and higher throughput in specific access patterns.

Document Stores (MongoDB, CouchDB)

Store data as JSON-like documents (BSON in MongoDB). Each document is self-contained and can have a different schema. Excellent for hierarchical, nested data.

```json
// MongoDB document - embedded approach
{
  "_id": "order_123",
  "user_id": "u_456",
  "items": [
    { "product_id": "p_1", "qty": 2, "price": 29.99 },
    { "product_id": "p_2", "qty": 1, "price": 49.99 }
  ],
  "total": 109.97,
  "status": "delivered"
}
```

Use when: Content management, product catalogs, user profiles, real-time analytics where data is naturally document-shaped and schemas evolve frequently.

Embedded vs Referenced documents: Embed related data when it is always accessed together and doesn't grow unboundedly (order items embedded in order). Reference (use foreign keys) when the related data is large, shared, or accessed independently.

Key-Value Stores (Redis, DynamoDB, Riak)

The simplest NoSQL model — store and retrieve values by key. Extremely fast reads and writes at massive scale.

Use when: Session storage, user preferences, shopping carts, distributed caching, leaderboards (Redis Sorted Sets), feature flags.

Hot keys problem: In DynamoDB or Redis Cluster, if one key receives a disproportionate number of requests (e.g., a celebrity's user record), it becomes a hotspot. Solutions: add a random suffix to the key and distribute across multiple shards, or cache at a higher level.

Wide-Column Stores (Cassandra, HBase)

Data is organized into tables, rows, and column families, but unlike relational databases, each row can have different columns. Designed for massive write throughput and time-series data.

Cassandra uses a **partition key** to distribute rows across nodes and a **clustering key** to sort rows within a partition. Data is written to an in-memory structure (memtable) and periodically flushed to disk (SSTable).

```
Cassandra data model for messages:
Table: messages
Partition key: conversation_id (determines which node stores the row)
Clustering key: message_timestamp DESC (sorts messages newest first)

Row: conversation_id=C1 | t=100 | "Hello"
Row: conversation_id=C1 | t=99  | "How are you?"
Row: conversation_id=C2 | t=150 | "Hi"
```

Use when: Time-series data (IoT sensor readings, logs), messaging systems, recommendation engines, write-heavy workloads with predictable access patterns.

Graph Databases (Neo4j, Amazon Neptune)

Store data as nodes (entities) and edges (relationships), both of which can have properties. Optimized for traversing complex relationship networks.

```
Graph: Social network
(Alice)-[:FRIENDS_WITH]->(Bob)-[:FRIENDS_WITH]->(Charlie)
(Alice)-[:LIKES]->(Post1)
(Bob)-[:AUTHORED]->(Post1)
```

Use when: Social networks, fraud detection (finding connected fraudulent accounts), knowledge graphs, recommendation engines ("friends who bought X also bought Y"), network topology.

3.3 SQL vs NoSQL Comprehensive Comparison

Dimension	SQL (Relational)	NoSQL
Schema	Fixed, predefined schema	Dynamic or schema-less
Consistency	ACID transactions	BASE (Eventually consistent)
Scaling	Primarily vertical	Horizontal (designed for it)
Query language	SQL (standardized)	DB-specific APIs
Relationships	Foreign keys, JOINs	Denormalized, application-side
Transactions	Multi-row ACID	Limited (single-document in MongoDB)
Use cases	Financial systems, ERP, complex queries	Web apps, real-time, IoT, unstructured data
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, Redis, DynamoDB
Maturity	50+ years, battle-tested	10-15 years
Indexes	B+ Tree, full-text, spatial	Varies by DB

Choose SQL when:

- You need ACID transactions (payments, inventory, banking)
- Your data is highly relational (e.g., orders → products → suppliers)
- Schema is well-defined and stable
- Complex reporting queries with ad-hoc JOINs

Choose NoSQL when:

- You need to scale writes horizontally beyond what a single master can handle
- Data is unstructured or semi-structured and evolves frequently
- Access patterns are simple and well-defined (key lookups, time-series queries)

- You need very low latency at high throughput (cache replacement, session store)

3.4 Time-Series Databases

Time-Series Databases (TSDBs) are optimized for storing and querying data that is indexed by time — metrics, sensor readings, financial ticks, log events.

Key characteristics: Data is append-only (past data never changes), queries are almost always time-range based, compression is highly effective (values change slowly), and data has a natural retention policy (delete data older than 90 days).

InfluxDB: Purpose-built TSDB with a SQL-like query language (InfluxQL/Flux). Stores data in "measurements" (like tables) with tags (indexed metadata) and fields (values). Excellent for DevOps metrics.

TimescaleDB: PostgreSQL extension that adds automatic time-based table partitioning and TSDB-specific functions while retaining full SQL compatibility. Best choice if you need SQL on time-series data.

Use when: Infrastructure monitoring (CPU, memory over time), IoT sensor streams, financial market data, application performance metrics, anomaly detection.

3.5 Blob/Object Storage

Object storage (Amazon S3, Google Cloud Storage, Azure Blob) stores unstructured data — images, videos, documents, backups — as objects with metadata and a unique key, in a flat namespace (no directories, though key prefixes simulate folders).

Key features:

- **Multipart upload:** Large files (>5 GB) are split into parts and uploaded in parallel, then assembled. Each part can be individually retried on failure.
- **Pre-signed URLs:** Temporary URLs that grant time-limited access to a private object without exposing credentials. Used for client-side uploads directly to S3 (bypassing your servers).
- **Storage tiers:** S3 has Standard, Standard-IA (Infrequent Access), Glacier (archival) — optimize cost by lifecycle policies.

```
Direct Upload Pattern (preferred for large files):
Client --> Request pre-signed URL from App Server
       <-- Receives S3 pre-signed URL
Client --> Upload directly to S3 (bypasses app servers!)
S3     --> Notifies App Server via S3 Event / SQS
App Server --> Process uploaded file (thumbnail, transcoding, etc.)
```

3.6 Data Warehouse vs Data Lake

Dimension	Data Warehouse	Data Lake
Data type	Structured, processed	Raw, all formats (structured + unstructured)
Schema	Schema-on-write (defined before loading)	Schema-on-read (applied when querying)
Processing	ETL (Extract, Transform, Load)	ELT (Extract, Load, Transform)
Users	Business analysts, BI tools	Data scientists, ML engineers
Query pattern	OLAP (complex aggregations, reporting)	Exploratory analytics, ML training
Examples	Snowflake, BigQuery, Redshift	S3 + Athena, Azure Data Lake, Databricks
Cost	Higher (optimized storage + compute)	Lower storage, pay per query

OLAP vs OLTP:

- **OLTP** (Online Transaction Processing): High-volume, low-latency, simple read/write operations. Row-oriented storage. MySQL, PostgreSQL.
- **OLAP** (Online Analytical Processing): Complex queries over large datasets, aggregations, historical analysis. Column-oriented storage (reads only needed columns). Redshift, BigQuery, ClickHouse.

3.7 Search Databases

Elasticsearch and **Solr** are distributed search engines built on Apache Lucene. They

create an **inverted index** — a mapping from terms to the documents containing them — enabling full-text search in milliseconds.

```
Inverted Index:
Documents:
  D1: "the quick brown fox"
  D2: "the lazy dog"
  D3: "quick lazy fox"

Inverted Index:
"quick" → [D1, D3]
"lazy"  → [D2, D3]
"fox"   → [D1, D3]
```

Query for "quick fox" → intersect [D1, D3] ∩ [D1, D3] = [D1, D3], ranked by relevance score.

Use when: Product search, log analysis (ELK Stack for Elasticsearch), full-text search in documents, autocomplete, fuzzy matching.

Architecture: Documents are divided into **shards** (for horizontal scale) and each shard has **replicas** (for fault tolerance). Queries are distributed across shards and results aggregated.

Chapter 3 — Quick Revision Box

> - **ACID:** Atomicity (all-or-nothing), Consistency (valid state transitions), Isolation (concurrent = serial), Durability (commits survive crashes)
> - **B+ Tree index:** Leaf nodes store data, linked for range scans; composite index order matters (leftmost prefix rule)
> - **Document DB:** Nested JSON, flexible schema (MongoDB); **Key-Value:** Ultra-fast lookups (Redis, DynamoDB); **Wide-Column:** Time-series writes (Cassandra); **Graph:** Relationship traversal (Neo4j)
> - **Choose SQL** for complex relationships, ACID, stable schema; **Choose NoSQL** for horizontal write scale, flexible schema, high-throughput simple queries
> - **Sharding** distributes different rows to different servers; **replication** copies same data to multiple servers
> - **Object storage** (S3): Use pre-signed URLs for direct client uploads; multipart for large files
> - **OLAP** (column-oriented, analytical) ≠ **OLTP** (row-oriented, transactional); search engines use **inverted indexes**

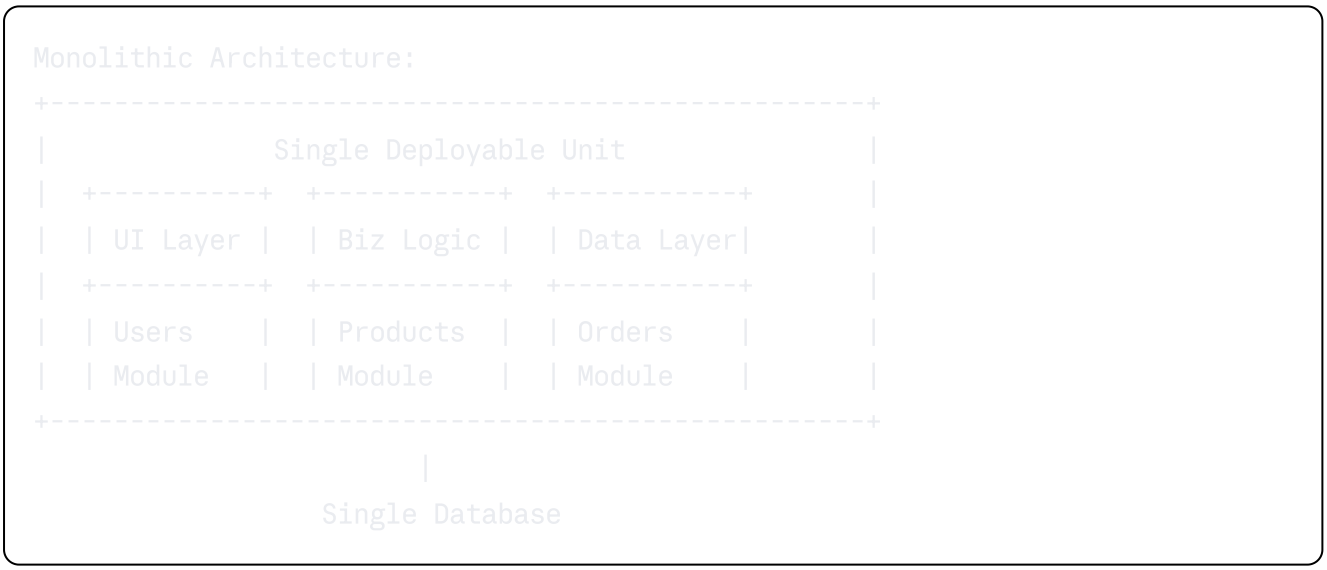
Chapter 4: Microservices & System Architecture {#chapter-4}

4.1 Monolithic Architecture

A **monolith** is a single deployable unit containing all application functionality — the UI, business logic, and data access layers are packaged together and run as a single process.

Advantages: Simple to develop initially, easy to test end-to-end, no network overhead between modules, simple deployment (deploy one artifact), easy debugging (single process, single log).

Disadvantages: Scaling requires scaling the entire application (can't scale individual features), long build and test cycles as the codebase grows, technology lock-in (entire app must use same language/framework), a bug in one module can crash the entire application, deployment is all-or-nothing (risky).



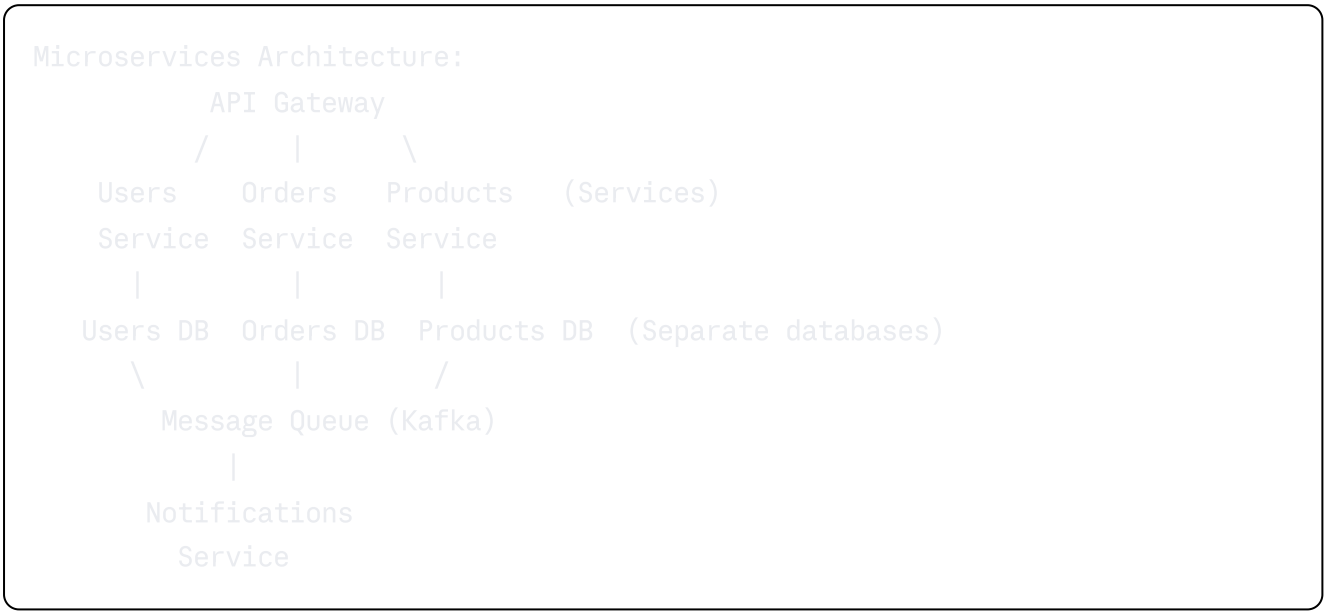
When monolith is appropriate: Small teams (2-5 developers), early-stage products with evolving requirements, simple applications with low traffic, when the team lacks distributed systems expertise.

4.2 Microservices Architecture

Microservices decompose an application into a collection of small, independently deployable services, each responsible for a specific business capability and communicating via APIs.

Key characteristics:

- Each service is independently deployable and scalable
- Services own their own database (database-per-service pattern)
- Services communicate via REST, gRPC, or message queues
- Each service can be built with a different technology stack
- Services are organized around business capabilities, not technical layers



Advantages: Independent scaling (scale Orders but not Users), technology flexibility, smaller codebases (easier to understand and modify), fault isolation, faster release cycles per service.

Disadvantages: Distributed systems complexity (network failures, partial failures), data consistency challenges (no cross-service ACID transactions), service discovery overhead, operational complexity (monitoring many services), higher latency (network calls replace in-process calls).

4.3 Monolith vs Microservices Comparison

Dimension	Monolith	Microservices
Deployment	Single artifact	Independent per service
Scaling	Scale entire app	Scale per service
Technology	Single stack	Polyglot (per service)
Data	Shared database	Database per service
Communication	In-process method calls	Network calls (REST, gRPC, MQ)

Dimension	Monolith	Microservices
Development speed (early)	Fast	Slower (infrastructure overhead)
Development speed (mature)	Slower (coordination)	Faster (independent teams)
Testing	Simpler	Complex (integration tests)
Debugging	Easy (single log)	Hard (distributed tracing needed)
Failure isolation	No (one bug, full outage)	Yes (service-level failures)
Consistency	Easy (single DB)	Hard (eventual consistency)
Best for	Small teams, early stage	Large teams, high scale

4.4 Service Discovery

When services need to communicate, they need to find each other's network addresses. In dynamic environments (containers, cloud), IP addresses change constantly — **service discovery** solves this.

Client-side discovery: The client queries a service registry (Eureka, Consul) to get the list of available instances and applies client-side load balancing (Netflix Ribbon does this). *Advantage:* flexible load balancing. *Disadvantage:* registry client code in every service.

Server-side discovery: The client makes a request to a load balancer or DNS name. The infrastructure (AWS ALB, Kubernetes DNS, API Gateway) resolves the name to an instance. *Advantage:* simpler clients. *Disadvantage:* additional network hop.

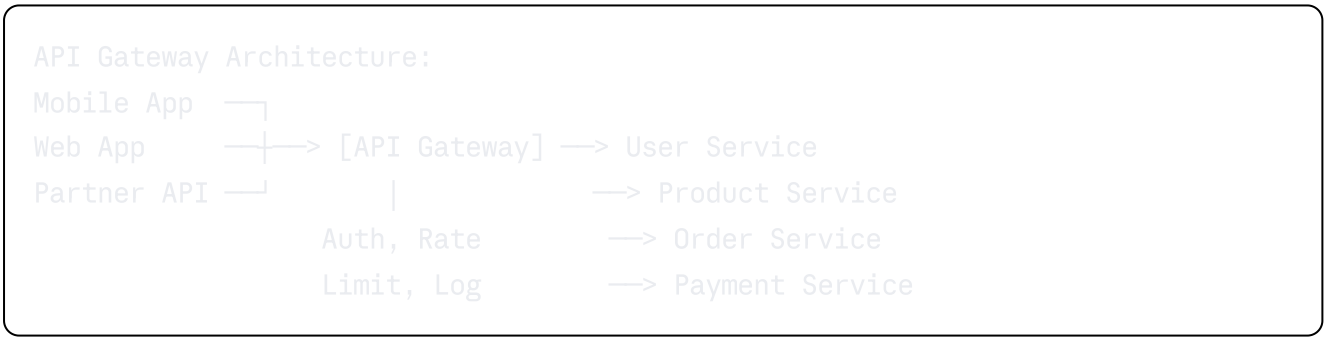
```
Client-side discovery:
Client → Eureka/Consul Registry: "Where is UserService?"
       ← [192.168.1.1:8080, 192.168.1.2:8080]
Client → Load balance and call directly

Server-side discovery:
Client → "userservice.internal:8080"
       → DNS/Load Balancer resolves
       → Routes to available instance
```

4.5 API Gateway

An **API Gateway** is the single entry point for all client requests. It acts as a reverse proxy, routing requests to appropriate microservices while providing cross-cutting concerns at the infrastructure level.

API Gateway functions:

- **Request routing:** Route `/api/users` to User Service, `/api/orders` to Order Service
- **Authentication & Authorization:** Validate JWT tokens once at the gateway instead of in every service
- **Rate limiting:** Enforce per-client or per-endpoint limits
- **SSL/TLS termination:** Handle HTTPS at the gateway; backend services use plain HTTP
- **Request/Response transformation:** Convert between formats (e.g., REST to gRPC)
- **Request aggregation:** Combine multiple service calls into a single client response
- **Caching:** Cache responses for read-heavy endpoints
- **Logging & monitoring:** Centralized access logs



Popular API Gateways: Kong (open-source, Nginx-based), AWS API Gateway (managed), Apigee (Google), Zuul (Netflix), Traefik (cloud-native).

4.6 Service Mesh

A **service mesh** is a dedicated infrastructure layer that handles service-to-service communication. Instead of embedding network logic in application code, it uses **sidecar proxies** (Envoy) deployed alongside each service container.

Data plane: The sidecar proxies that intercept and handle all network traffic. (Envoy)
Control plane: Centralized management for configuring the proxies. (Istio Pilot, Linkerd control plane)

Features provided by service mesh: mTLS (automatic mutual TLS between services), circuit breaking, retries, timeouts, observability (distributed tracing, metrics), traffic shaping (canary deployments, A/B testing).

4.7 Inter-Service Communication

Synchronous Communication

REST: HTTP-based, stateless, widely understood, tooling-rich. Best for public APIs and simple request-response patterns. Downside: tight coupling — if the downstream service is slow, the upstream service waits.

gRPC: Google's RPC framework using HTTP/2 and Protocol Buffers (binary serialization). ~10× more efficient than JSON over REST for high-throughput internal service communication. Supports bidirectional streaming. Requires protobuf schema definition.

```protobuf
// gRPC proto definition
service UserService {
  rpc GetUser(GetUserRequest) returns (User);
  rpc StreamUsers(Empty) returns (stream User);
}
```

GraphQL: Client specifies exactly which fields it needs; the server returns only those fields. Reduces over-fetching (getting more data than needed) and under-fetching (needing multiple requests for related data). Best for client-facing APIs with varying data needs.

REST vs gRPC vs GraphQL

Feature	REST	gRPC	GraphQL
Protocol	HTTP/1.1	HTTP/2	HTTP/1.1 or 2
Format	JSON/XML	Protocol Buffers (binary)	JSON
Performance	Good	Excellent	Good
Streaming	Polling/SSE	Bidirectional	Subscriptions
Schema	OpenAPI (optional)	.proto (required)	GraphQL schema (required)
Browser support	Native	Limited (grpc-web)	Native
Best for	Public APIs	Internal microservices	Client-facing with varying needs
Tooling	Excellent	Good	Good

Asynchronous Communication

Services communicate by publishing events to a message broker (Kafka, RabbitMQ). The publisher doesn't wait for a response — it fires and moves on. This decouples services temporally and improves resilience.

4.8 Backends for Frontends (BFF)

The **BFF pattern** creates separate backend services for different frontend clients (mobile app, web app, partner API), each tailored to that client's specific data needs.

Problem: A single general-purpose API returns all fields. The mobile app only needs 20% of the data, wastes bandwidth, and requires extra processing. *Solution:* Mobile BFF returns only mobile-needed data; Web BFF aggregates and formats for web.

4.9 The Saga Pattern

The **Saga pattern** handles distributed transactions across multiple microservices. Since you can't use a single database ACID transaction across services, you instead

break it into a sequence of local transactions, each publishing an event to trigger the next.

If a step fails, **compensating transactions** undo the previous steps.

Choreography-based Saga: Services react to events without a central coordinator. Decoupled but harder to track. **Orchestration-based Saga:** A central Saga Orchestrator tells each service what to do and handles failures. Easier to monitor and debug.

```
Order Saga (Orchestration):
Orchestrator → Create Order Service: "Create Order"
            ← OrderCreated event
Orchestrator → Payment Service: "Charge Payment"
            ← PaymentFailed event
Orchestrator → Order Service: "Cancel Order" (compensating transaction)
```

4.10 CQRS & Event Sourcing

CQRS (Command Query Responsibility Segregation): Separates the data model for reads (queries) from the data model for writes (commands). Allows independent scaling and optimization of each path.

```
CQRS Architecture:
Command (Write):              Query (Read):
Client → Command Handler      Client → Query Handler
       → Write DB (normalized)       → Read DB (denormalized/view)
            |                                  ↑
            +── Event → Projection ──────────+
```

Event Sourcing: Instead of storing the *current state* of an entity, store a sequence of *events* that led to the current state. The current state is derived by replaying all events.

Advantages: Complete audit log, ability to replay events to derive any past state, natural fit with CQRS. *Disadvantages:* Querying current state requires replaying events (mitigated with snapshots), eventual consistency.

```
Event Store for a bank account:
Event 1: AccountOpened { balance: 0 }
Event 2: MoneyDeposited { amount: 1000 }
Event 3: MoneyWithdrawn { amount: 200 }
Event 4: MoneyDeposited { amount: 500 }

Current balance = replay: 0 + 1000 - 200 + 500 = 1300
```

Chapter 4 — Quick Revision Box

> - **Monolith:** Simple to start, hard to scale; all-or-nothing deployment; one codebase, one DB
> - **Microservices:** Independent scaling and deployment; database-per-service; needs service discovery and API gateway
> - **API Gateway:** Single entry point; handles auth, rate limiting, SSL termination, routing, aggregation
> - **Service Mesh (Istio):** Sidecar proxy (Envoy) handles cross-cutting concerns (mTLS, retries, tracing) transparently
> - **Saga pattern:** Handles distributed transactions; choreography (decoupled events) vs orchestration (central coordinator)
> - **CQRS:** Separate read and write models; allows independent optimization and scaling
> - **Event Sourcing:** Store events, not state; enables complete audit log and time-travel queries

---

Chapter 5: Message Queues & Event-Driven Architecture {#chapter-5}

5.1 Message Queue Basics

A **message queue** is an asynchronous communication mechanism where producers send messages to a queue and consumers receive them independently. The queue acts as a **buffer**, decoupling producers from consumers.

Core components:

- **Producer:** Service that sends messages
- **Consumer:** Service that receives and processes messages

- **Broker:** The middleware that stores and routes messages (Kafka, RabbitMQ)
- **Topic/Queue:** Named destination for messages
- **Message:** The payload being transmitted (JSON, Avro, Protobuf)

Benefits: Decoupling (producers and consumers are independent), buffering (absorbs traffic spikes), load leveling (consumers process at their own pace), durability (messages persisted on broker disk).

5.2 Point-to-Point vs Publish-Subscribe

Point-to-Point (Queue): One producer sends a message; exactly one consumer processes it. Like a task queue — once processed, the message is gone. Used for work distribution (order processing, email sending).

Publish-Subscribe (Topic): One producer publishes a message to a topic; all subscribed consumers receive a copy. Used for event broadcast (a new order event triggers both the email service and the analytics service).

```
Point-to-Point:
Producer → [Queue] → Consumer 1 (processes message, message deleted)

Pub/Sub:
Producer → [Topic] → Consumer 1 (inventory)
                   → Consumer 2 (email notification)
                   → Consumer 3 (analytics)
```

5.3 Message Delivery Semantics

At-most-once: Message is delivered zero or one time. Sent without acknowledgment — if delivery fails, the message is lost. Lowest latency, highest risk of data loss. *Use case:* Metrics where losing occasional data points is acceptable.

At-least-once: Message is delivered one or more times. Producer retries until it receives an acknowledgment. Consumer may receive duplicates. *Use case:* Most business events — implement idempotent consumers to handle duplicates.

Exactly-once: Message is delivered exactly once. Hardest to achieve — requires both idempotent producers and transactional consumers. Kafka supports exactly-once semantics (EOS) via producer idempotence and transactional APIs.

Semantic	Duplicates?	Data Loss?	Latency	Complexity
At-most-once	No	Possible	Lowest	Simple
At-least-once	Possible	No	Medium	Medium
Exactly-once	No	No	Highest	Complex

5.4 Message Brokers Comparison

Feature	Apache Kafka	RabbitMQ	AWS SQS	Google Pub/Sub
Type	Distributed log	Traditional AMQP broker	Managed queue service	Managed pub/sub
Throughput	Millions/sec	~50K/sec	Moderate	High
Retention	Configurable (days/forever)	Until consumed	4 days (default, max 14)	7 days
Ordering	Per partition	Per queue	Per FIFO queue	Per ordering key
Consumer model	Pull (consumer controls)	Push/Pull	Pull (long polling)	Push/Pull
Replay	Yes (seek to offset)	No	No	Limited
Exactly-once	Yes (EOS)	No	No	No
Use case	Event streaming, log aggregation	Task queues, RPC	Simple queues, AWS native	Google Cloud native

5.5 Apache Kafka In Depth

Kafka is a distributed event streaming platform designed for high-throughput, fault-tolerant, ordered message delivery.

Core Concepts

Topic: A named stream of records. Like a table in a database, but for events. A topic has one or more **partitions**.

Partition: A topic is split into N partitions for horizontal scalability. Each partition is an ordered, immutable sequence of records. Partitions are distributed across broker nodes.

Offset: Each record in a partition has a unique sequential ID called an offset. Consumers track their position by committing offsets.

Consumer Group: A group of consumers that jointly consume a topic. Each partition is consumed by exactly one consumer within a group. To scale consumption, add more consumers to a group (up to the number of partitions).

```
Kafka Topic with 3 Partitions, 2 Consumer Groups:

Topic: orders
  Partition 0: [msg0, msg1, msg2, msg3, msg4]
  Partition 1: [msg0, msg1, msg2]
  Partition 2: [msg0, msg1, msg2, msg3]

Consumer Group A (Payment Service):
  Consumer A1 → Partition 0
  Consumer A2 → Partition 1
  Consumer A3 → Partition 2

Consumer Group B (Analytics Service):
  Consumer B1 → Partition 0 + 1
  Consumer B2 → Partition 2
```

Replication and Fault Tolerance

Each partition has one **leader** and N-1 **followers** (replicas). Producers and consumers interact only with the leader. Followers replicate data from the leader. If the leader fails, one of the **In-Sync Replicas (ISR)** is elected as the new leader.

ISR (In-Sync Replicas): The set of replicas that are fully caught up with the leader. A replica is removed from ISR if it falls too far behind. Producers can set `acks=all` to only acknowledge after all ISR replicas receive the message — strongest durability guarantee.

Log Compaction

By default, Kafka retains messages for a time window (7 days). With **log compaction**, Kafka retains the *latest value* for each key indefinitely — old values with the same key

are deleted. Used for maintaining the "current state" of entities.

5.6 Dead Letter Queues (DLQ)

A **Dead Letter Queue** is a special queue where messages that cannot be processed successfully are redirected after exceeding the maximum retry count. This prevents a "poison pill" message from blocking the entire queue.

```
Normal Flow:
Producer → [Queue] → Consumer → Process (Success)

Failure Flow:
Producer → [Queue] → Consumer → Process (Fail) → Retry 3 times → DLQ
                                                                  ↓
                                                    Alert + Manual inspection
```

DLQs enable: debugging failed messages, replaying after fixing the consumer bug, monitoring for anomalies.

5.7 Message Ordering

Kafka guarantees ordering **within a partition**. To guarantee order for related messages (e.g., all events for the same user), route them to the same partition using the user_id as the partition key.

```
Order guarantee: use consistent partition key
produce(key="user_123", value=OrderCreated)  → Partition 1
produce(key="user_123", value=OrderShipped)  → Partition 1 (same key!)
produce(key="user_456", value=OrderCreated)  → Partition 2 (different key)
```

Global ordering across all partitions is not possible at scale without sacrificing throughput.

Chapter 5 — Quick Revision Box

> - **Message queues** decouple producers and consumers; provide buffering, durability, and load leveling
> - **Point-to-Point:** one consumer per message; **Pub/Sub:** all subscribers get a copy

- **Delivery semantics:** at-most-once (fast, lossy) → at-least-once (most common, needs idempotent consumer) → exactly-once (hardest)
- **Kafka beats traditional MQ** for: replay capability, massive throughput, consumer group model, log retention
- **Kafka partitions:** unit of parallelism; #consumers in group ⩽ #partitions for efficiency
- **ISR (In-Sync Replicas):** set of fully-caught-up replicas; `acks=all` ensures durability
- **DLQ (Dead Letter Queue):** catch failed messages after max retries; essential for production systems

---

Chapter 6: Security & Authentication in Systems {#chapter-6}

6.1 Authentication vs Authorization

Authentication (AuthN): Verifying *who you are*. Proving your identity. "Are you really Alice?" **Authorization** (AuthZ): Verifying *what you can do*. Checking permissions. "Alice, can you delete this resource?"

Authentication always precedes authorization. You cannot determine permissions without first verifying identity.

6.2 Session-Based Authentication

The traditional web authentication model:

1. User submits credentials (username/password)
2. Server validates and creates a **session** in the session store (database, Redis)
3. Server returns a **session ID** in a cookie
4. Subsequent requests include the cookie; server looks up the session to authenticate

Problem with horizontal scaling: If requests are load-balanced across servers, a request that hits Server B won't find a session created by Server A. Solutions: sticky sessions (bad for scaling), or centralized session store (Redis — all servers share the same session store).

6.3 Token-Based Authentication (JWT)

JSON Web Token (JWT) is a self-contained, signed token that encodes the user's identity and claims without requiring a server-side session store.

JWT Structure: Three base64-encoded parts separated by dots: `header.payload.signature`

```
Header:  { "alg": "HS256", "typ": "JWT" }
Payload: { "sub": "user_123", "email": "alice@example.com", "exp": 1719000000
}
Signature: HMAC_SHA256(base64(header) + "." + base64(payload), secret_key)
```

The server validates the JWT by verifying the signature with the secret key — no database lookup needed.

Access Token vs Refresh Token:

- **Access token:** Short-lived (15 minutes). Used for API calls. Stored in memory (not localStorage — XSS risk).
- **Refresh token:** Long-lived (7-30 days). Used only to obtain new access tokens. Stored in an HttpOnly cookie (not accessible to JavaScript).

```
Token refresh flow:
Access token expires → Client sends refresh token to /refresh
Server → Validate refresh token → Issue new access token + new refresh token
       → Old refresh token invalidated (rotation)
```

JWT tradeoff: Stateless (no server-side storage), but cannot be invalidated before expiry. Solution: maintain a token blocklist in Redis, or use very short expiry times.

6.4 OAuth 2.0

OAuth 2.0 is an authorization framework that allows a third-party application to obtain limited access to a user's account on a service, without exposing the user's credentials to the third party.

Authorization Code Flow (Most Secure — for Web Apps)

```
1. User clicks "Login with Google" on your app
2. App redirects to Google's authorization server:
   GET /auth?client_id=APP_ID&redirect_uri=https://yourapp.com/callback
            &response_type=code&scope=email profile&state=random_csrf_token
3. User authenticates with Google and grants permissions
4. Google redirects to your app: GET /callback?
code=AUTH_CODE&state=random_csrf_token
5. App exchanges code for tokens (server-side, code is one-time use):
   POST /token {code, client_id, client_secret, redirect_uri}
6. Google returns: { access_token, refresh_token, expires_in }
7. App uses access_token to call Google APIs on behalf of user
```

PKCE (Proof Key for Code Exchange): Extension for mobile/SPA apps (no client secret). The app generates a `code_verifier`, sends its hash (`code_challenge`) in step 2, and sends the original `code_verifier` in step 5 — Google verifies they match, preventing authorization code interception attacks.

6.5 HTTPS/TLS

TLS (Transport Layer Security) encrypts data in transit, preventing eavesdropping and man-in-the-middle attacks. HTTPS = HTTP over TLS.

mTLS (Mutual TLS): Both client and server authenticate each other using certificates. Used for service-to-service communication in microservices (service mesh). Ensures not just "is this the real server?" but also "is this an authorized client?"

6.6 Data Encryption

Encryption at rest: Data stored on disk is encrypted. AES-256 is the standard. Protects against physical theft of storage media. Databases (RDS), object stores (S3), and disks (EBS) all support server-side encryption.

Encryption in transit: Data moving over the network is encrypted via TLS. Always use HTTPS for external APIs; use mTLS for internal service communication.

Field-level encryption: Individual sensitive fields (SSN, credit card number) are encrypted before storage, even within an encrypted database. Only services with the decryption key can read the value. Provides an additional layer of protection against database breaches.

6.7 Common Vulnerabilities & Prevention

SQL Injection: Attacker injects malicious SQL into user input. Prevention: use parameterized queries (prepared statements), never concatenate user input into SQL strings.

```java
// VULNERABLE:
query = "SELECT * FROM users WHERE name = '" + username + "'";
// Input: admin'; DROP TABLE users; --

// SAFE (Parameterized query):
PreparedStatement stmt = conn.prepareStatement("SELECT * FROM users WHERE name =
stmt.setString(1, username);
```

XSS (Cross-Site Scripting): Attacker injects malicious JavaScript that runs in other users' browsers. Prevention: escape/sanitize user-generated HTML, use Content-Security-Policy headers, HttpOnly cookies.

CSRF (Cross-Site Request Forgery): Attacker tricks a user's browser into making unintended requests to a site where the user is authenticated. Prevention: CSRF tokens (validate a random token in POST requests), SameSite cookie attribute, check Origin/Referer headers.

6.8 Secret Management

Never hardcode secrets (API keys, database passwords, private keys) in source code or environment variable files committed to version control.

Solution	Description	Best For
HashiCorp Vault	Centralized secrets management with dynamic secrets and audit logs	Complex enterprise environments
AWS Secrets Manager	Managed secret store with automatic rotation	AWS-native applications
AWS Parameter Store	Simpler, cheaper secret storage (SSM)	Simpler configs and non-rotated secrets
Kubernetes Secrets	Base64-encoded (not encrypted by default!) secrets in k8s	Container workloads (use with Vault or SOPS)
Environment variables	Runtime injection of secrets	Never in code; acceptable as final delivery mechanism

---

Chapter 6 — Quick Revision Box

> - **AuthN** (who are you?) precedes **AuthZ** (what can you do?)
> - **Session:** server-side state; scale via centralized Redis session store; sticky sessions are an anti-pattern
> - **JWT:** stateless tokens; access (short-lived, in-memory) + refresh (long-lived, HttpOnly cookie); cannot be revoked without blocklist
> - **OAuth 2.0:** third-party authorization; Authorization Code + PKCE is the gold standard for web/mobile
> - **mTLS:** both sides present certificates; used for service-to-service auth in service meshes
> - **Encryption:** at-rest (AES-256) + in-transit (TLS) + field-level for ultra-sensitive data
> - **SQL Injection:** always use parameterized queries; **CSRF:** use CSRF tokens + SameSite cookies

---

Chapter 7: Monitoring, Logging & Observability {#chapter-7}

7.1 The Three Pillars of Observability

Observability is the ability to understand the internal state of a system by examining

its external outputs. The three pillars are:

Metrics: Numerical measurements over time (CPU usage at 80%, request latency P99 = 250ms, error rate = 0.1%). Low storage cost; perfect for dashboards, alerting, and capacity planning.

Logs: Timestamped records of discrete events ("2024-01-15 10:23:45 ERROR Payment failed for order_id=123"). High granularity, searchable, but expensive to store at scale.

Traces: End-to-end journey of a single request through multiple services, showing which service took how long. Essential for debugging latency issues in microservices.

```
Three Pillars:
[Metrics]  → Are systems healthy? (dashboard, alerts)
[Logs]     → What happened? (debugging, audit)
[Traces]   → Why is it slow? (latency attribution)
```

7.2 Logging Best Practices

Structured logging: Log in JSON format instead of plain text. Machine-parseable, enables powerful querying.

```json
// Unstructured (hard to parse):
"2024-01-15 10:23 ERROR Payment failed for user 123 order 456 amount 99.99"

// Structured (queryable):
{
  "timestamp": "2024-01-15T10:23:45Z",
  "level": "ERROR",
  "service": "payment-service",
  "event": "payment_failed",
  "user_id": "123",
  "order_id": "456",
  "amount": 99.99,
  "error": "insufficient_funds",
  "trace_id": "abc123"
}
```

Log levels (least to most verbose): FATAL → ERROR → WARN → INFO → DEBUG → TRACE

Log aggregation (ELK Stack):

- **E**lasticsearch: Stores and indexes logs
- **L**ogstash (or Fluentd/Filebeat): Collects and ships logs
- **K**ibana: Visualizes and queries logs

7.3 Distributed Tracing

In a microservices system, a single user request might touch 10 services. Traditional logs can't show the end-to-end flow. **Distributed tracing** assigns a unique **trace ID** to each request and propagates it through all service calls.

```
Trace for GET /checkout:
Trace ID: xyz789
|
+-- [API Gateway]          0ms → 5ms
|   |
|   +-- [Cart Service]     5ms → 45ms
|   |   |
|   |   +-- [Redis Cache]  6ms → 8ms (HIT)
|   |
|   +-- [Payment Service]  45ms → 180ms (SLOW!)
|       |
|       +-- [Stripe API]   50ms → 175ms ← BOTTLENECK
```

Each unit of work within a trace is a **span**. Spans have a parent-child relationship. The trace visualization (waterfall diagram) immediately shows where latency is occurring.

Tools: OpenTelemetry (vendor-neutral instrumentation SDK), Jaeger (open-source by Uber), Zipkin (open-source by Twitter), Datadog APM, AWS X-Ray.

7.4 SLI, SLO, SLA

SLI (Service Level Indicator): A specific, measurable metric that indicates how well a service is performing. Examples: availability (% of successful requests), latency (P99 response time), throughput (requests/second), error rate (% of failed requests).

SLO (Service Level Objective): The target value or range for an SLI that you commit to internally. Example: "Availability SLO: 99.9% of requests succeed" or "Latency SLO: P99 < 200ms over 30 days." SLOs are internally set goals.

SLA (Service Level Agreement): A contractual commitment between a service provider and customer, with defined consequences (credits, refunds) if SLOs are not met. The SLA is the external, legally binding version of the SLO.

```
SLO = 99.9% availability
Error Budget = 100% - 99.9% = 0.1% per month
Monthly error budget = 0.1% × 30 days × 24 hours × 60 min = 43.2 minutes of
downtime
```

Error budgets enable data-driven reliability decisions: if you've spent your error budget, freeze risky deployments; if you have budget remaining, it's safe to move fast.

7.5 Chaos Engineering

Chaos Engineering is the practice of deliberately injecting failures into a system to discover weaknesses before they cause unplanned outages.

Netflix's **Simian Army** includes:

- **Chaos Monkey:** Randomly terminates instances in production
- **Chaos Gorilla:** Simulates an entire availability zone outage
- **Latency Monkey:** Introduces artificial latency between services
- **Conformity Monkey:** Finds instances not adhering to best practices

The key insight: failure is inevitable. By practicing failure regularly (in controlled conditions), teams build more resilient systems and are less surprised when failures occur in the wild.

Chapter 7 — Quick Revision Box

> - **Three pillars:** Metrics (what is happening?), Logs (what happened?), Traces (why is it slow?)
> - **Structured logging (JSON):** machine-parseable, queryable by fields; use trace_id in every log line
> - **ELK Stack:** Elasticsearch (index) + Logstash/Filebeat (collect) + Kibana (visualize)
> - **Distributed tracing:** trace_id flows across service boundaries; spans show latency per service
> - **SLI** = metric, **SLO** = internal target, **SLA** = external contract with penalties
> - **Error budget** = 100% - SLO; once exhausted, freeze deployments; if surplus, move fast

- **Chaos engineering:** inject failures deliberately to find weaknesses; Chaos Monkey kills random instances

---

Chapter 8: Cloud & DevOps Concepts for System Design {#chapter-8}

8.1 Cloud Service Models

Model	What You Manage	What Provider Manages	Examples
IaaS (Infrastructure as a Service)	OS, runtime, apps, data	Hardware, network, virtualization	AWS EC2, GCP Compute Engine, Azure VMs
PaaS (Platform as a Service)	Apps, data	Everything below (OS, runtime, infra)	AWS Elastic Beanstalk, Heroku, Google App Engine
SaaS (Software as a Service)	Configuration only	Everything	Gmail, Salesforce, Slack
FaaS (Function as a Service) / Serverless	Code only	Everything including server management	AWS Lambda, Google Cloud Functions, Azure Functions

---

8.2 Containers & Kubernetes

Docker packages an application and all its dependencies into a portable **container** — a lightweight, isolated execution environment. Containers share the host OS kernel (unlike VMs, which include a full OS), making them faster to start and more resource-efficient.

Kubernetes (K8s) is a container orchestration platform that automates: deploying containers, scaling them up/down, self-healing (restarting crashed containers), rolling updates, and service discovery.

Key Kubernetes Concepts

Resource	Description
Pod	Smallest deployable unit; one or more containers sharing network and storage
Deployment	Manages a set of identical Pods (ReplicaSet); handles rolling updates
Service	Stable network endpoint for a set of Pods; load balances across Pods
Ingress	HTTP routing rules; external traffic → Services (TLS termination, path-based routing)
ConfigMap	Non-sensitive configuration data (environment-specific settings)
Secret	Sensitive data (passwords, API keys); base64 encoded (not encrypted by default)
StatefulSet	Like Deployment but for stateful apps (databases); stable network IDs, ordered rollout
HPA	Horizontal Pod Autoscaler; scales pod count based on CPU/memory/custom metrics
VPA	Vertical Pod Autoscaler; adjusts CPU/memory requests per pod
Cluster Autoscaler	Scales the number of worker nodes in the cluster

```
Kubernetes Request Flow:
External Traffic
      |
  [Ingress Controller] (Nginx Ingress)
      |
  [Service] (ClusterIP, load balances Pods)
      |
  [Pods] (your application containers)
      |
  [PersistentVolumeClaim] → [Storage]
```

8.3 Serverless / FaaS

Serverless means you write functions and the cloud provider manages all infrastructure — no servers to provision, patch, or scale.

Advantages: No idle cost (pay per invocation), auto-scales to zero, no operational overhead.

Disadvantages:

- **Cold start:** First invocation after idle period has higher latency (100ms–3s) due to container initialization
- **Execution time limits:** AWS Lambda max 15 minutes; not for long-running jobs
- **Vendor lock-in:** AWS Lambda code isn't portable to GCP Functions
- **No persistent state:** Each invocation is stateless; use external storage for state

Use cases: Event-driven processing (resize image on S3 upload), scheduled jobs (cron), IoT data ingestion, API backends with unpredictable traffic, webhooks.

8.4 CI/CD Pipeline

Continuous Integration (CI): Developers frequently merge code changes into a shared repository, triggering automated builds and tests. Catches integration issues early.

Continuous Delivery (CD): Every successful CI build results in an artifact that is ready to deploy to production. Deployment is manual (requires approval).

Continuous Deployment (CD): Every successful build is automatically deployed to production without human intervention.

```
CI/CD Pipeline:
Code Commit → [CI: Build + Unit Tests + Integration Tests + Security Scan]
            → [CD: Deploy to Staging → Automated E2E Tests → Approval Gate]
            → [CD: Deploy to Production (blue-green or canary)]
```

Deployment Strategies

Blue-Green Deployment: Maintain two identical production environments (Blue = current, Green = new version). Deploy to Green, test, then switch the load balancer to point to Green. Instant rollback by switching back. *Downside:* Requires 2× infrastructure cost.

```
Blue-Green:
Before: LB → [Blue (v1)] [Green (idle)]
Deploy:       [Blue (v1)] [Green (v2)] ← deploy new version here
Switch: LB → [Blue (idle)] [Green (v2)]
Rollback if needed: LB → [Blue (v1)]
```

Canary Deployment: Gradually roll out new version to a small percentage of users (e.g., 5%), monitor for errors, then increase to 100%. Slower than blue-green but reduces blast radius.

```
Canary:
Traffic: 95% → v1 servers | 5% → v2 servers (canary)
Monitor metrics...
If OK: 50/50 → 100% v2
If Bad: rollback to 100% v1
```

Feature Flags: Toggle features on/off at runtime without code deployment. Enable features for specific users (internal testing), regions, or traffic percentages.

8.5 Infrastructure as Code (IaC)

IaC means defining and provisioning infrastructure through machine-readable configuration files instead of manual processes.

- **Terraform (HashiCorp):** Cloud-agnostic, declarative; manages cloud resources across AWS, GCP, Azure
- **CloudFormation (AWS):** AWS-native IaC in JSON/YAML
- **Ansible:** Configuration management and provisioning; procedural approach; excellent for server configuration

Benefits: Infrastructure is version-controlled, reproducible, and auditable. "Git blame" for your servers.

8.6 DNS & Global Load Balancing

DNS (Domain Name System) resolves human-readable domain names to IP addresses. DNS records relevant to system design:

- **A record:** Domain → IPv4 address
- **AAAA record:** Domain → IPv6 address
- **CNAME:** Domain → another domain (alias)

- **MX:** Mail server routing
- **TTL (Time to Live):** How long DNS resolvers cache the record

GeoDNS / Global Server Load Balancing (GSLB): Returns different IP addresses based on the client's geographic location. Route53, Cloudflare, and Akamai support this. Used to direct users to the nearest datacenter.

Latency-based routing (Route 53): Routes requests to the AWS region that provides the lowest latency for the requesting client — not necessarily the geographically nearest.

Chapter 8 — Quick Revision Box

> - **IaaS (EC2):** you manage OS+; **PaaS (Heroku):** you manage code+data; **SaaS:** you configure; **FaaS (Lambda):** you write functions
> - **Kubernetes:** Pod (unit), Deployment (manages pods), Service (stable endpoint), Ingress (external routing), HPA (auto-scale pods)
> - **Cold start** is serverless's main latency concern; mitigate with provisioned concurrency (AWS) or minimum instances
> - **Blue-green:** instant rollback, 2× infra cost; **Canary:** gradual rollout, minimal blast radius
> - **CI:** automated build+test on every commit; **CD (Delivery):** ready-to-deploy on success; **CD (Deployment):** auto-deploy on success
> - **Feature flags:** decouple feature release from code deployment; enable for percentage of users
> - **GeoDNS / GSLB:** route users to nearest datacenter; Route53 latency-based routing routes to lowest-latency region

---

Chapter 9: Designing Specific Systems — HLD Deep Dives {#chapter-9}

9.1 Design a URL Shortener (TinyURL)

Requirements

Functional:

- Given a long URL, return a short URL (e.g., `tiny.url/abc123`)

- Redirect short URL to original long URL
- Optional: custom aliases, URL expiration, analytics (click count, referrer, geo)

Non-Functional:

- High availability: 99.99% uptime
- Read-heavy: 100:1 read-to-write ratio
- Low redirect latency: P99 < 10ms
- Unique short codes; no collisions

Back-of-Envelope Calculations

```
Write QPS: 100M new URLs/day ÷ 86,400 sec ≈ 1,160 writes/sec
Read QPS:  1,160 × 100 (read:write ratio) = 116,000 reads/sec

Storage per URL record: 100 bytes (URL) + 7 bytes (code) + metadata ≈ 500
bytes
Daily storage: 1,160 writes/sec × 86,400 sec × 500 bytes ≈ 50 GB/day
5-year storage: 50 GB × 365 × 5 ≈ 90 TB (with 3x replication: ~270 TB)

Cache: 20% of URLs → 80% traffic. Hot URLs: 1,160/sec × 86,400 × 20% = 20M
URLs
Cache size: 20M × 500 bytes = 10 GB (fits in Redis)
```

API Design

```
POST /shorten
  Request:  { "url": "https://long.example.com/path?q=1", "alias": "custom",
"expires_at": "2025-01-01" }
  Response: { "short_url": "https://tiny.url/abc1234", "expires_at": "2025-01-
01" }
  Auth:     Bearer token

GET /{code}
  Response: HTTP 301/302 Redirect → Location: https://long.example.com/path?
q=1
  (301 = permanent, cached by browser; 302 = temporary, goes through server
each time → use 302 for analytics)
```

Database Schema

sql

```sql
-- URL Mapping Table (write-once, read-many)
CREATE TABLE url_mappings (
    short_code    VARCHAR(10)   PRIMARY KEY,
    original_url  VARCHAR(2048) NOT NULL,
    user_id       BIGINT,
    created_at    TIMESTAMP     DEFAULT NOW(),
    expires_at    TIMESTAMP,
    click_count   BIGINT        DEFAULT 0
);

-- Index for reverse lookup (long URL → short code for deduplication)
CREATE INDEX idx_original_url ON url_mappings(MD5(original_url));
```

Architecture Diagram

Key Design Decisions

Short code generation — Hashing vs Base62:

- **MD5/SHA256 hash of URL → take first 7 characters:** Fast but risk of collision and not guaranteed unique
- **Base62 encoding of auto-increment ID:** `[0-9][a-z][A-Z]` = 62 chars. 7-char Base62 = 62^7 = 3.5 trillion combinations. Predictable (sequential IDs reveal popularity) — shuffle using bit manipulation
- **Globally unique sequence generator (Snowflake):** Best for distributed systems

Collision handling: Store MD5 hash of original URL; if hash exists, return existing short code (deduplication). For random codes, use CAS (Compare-and-Swap) on insert.

Analytics: Publish click events to Kafka asynchronously; consumer aggregates to a separate analytics database (ClickHouse). Never block the redirect path for analytics writes.

9.2 Design a Web Crawler

Requirements

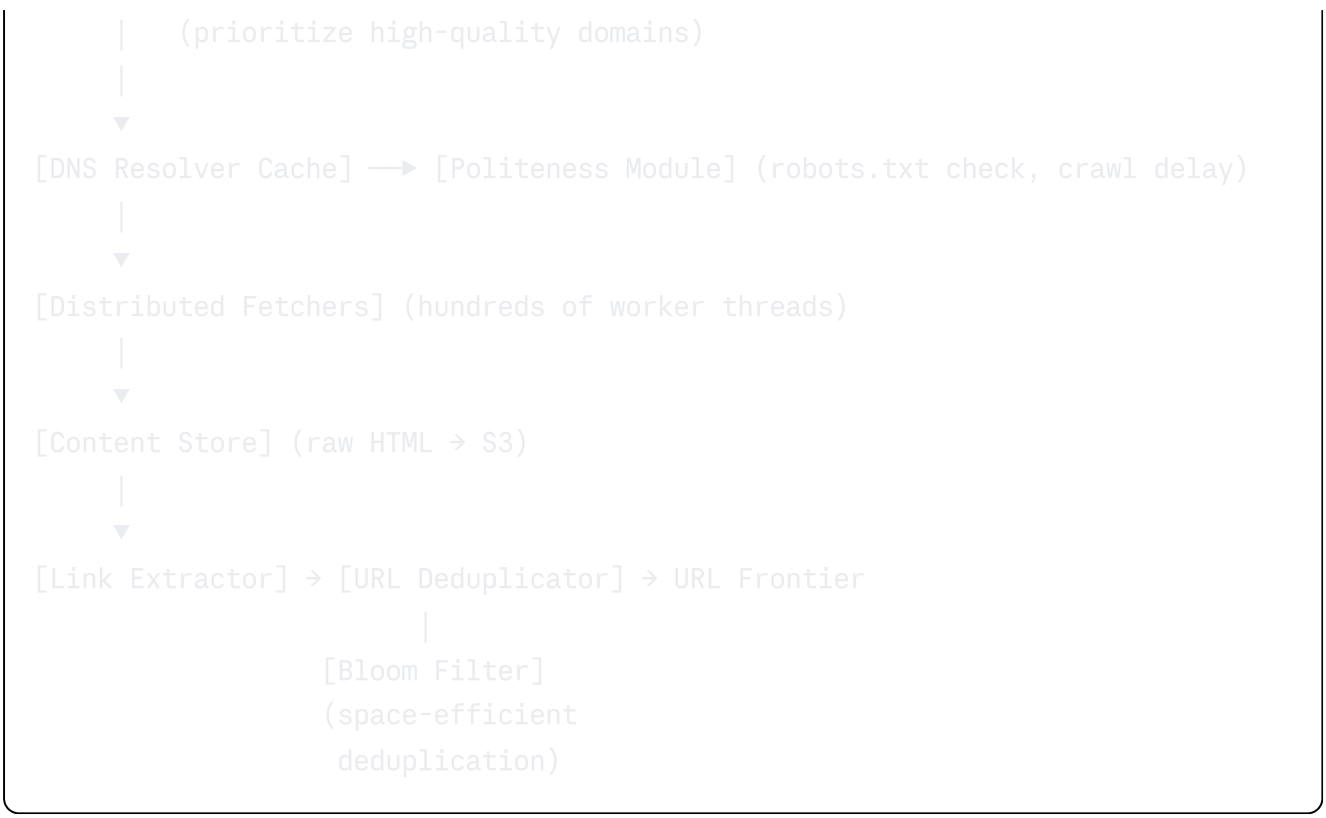
- Crawl billions of web pages, extract links, download content
- Politeness: respect robots.txt, don't overwhelm servers
- Deduplication: don't recrawl the same URL
- Scalability: 1 billion pages in 30 days

Back-of-Envelope Calculations

```
Pages to crawl: 1 billion
Time window: 30 days = 2,592,000 seconds
QPS needed: 1,000,000,000 / 2,592,000 ≈ 385 pages/second
Average page size: 100 KB
Download bandwidth: 385 × 100 KB = 38.5 MB/s
Storage: 1B × 100 KB = 100 TB per crawl cycle
```

High-Level Architecture

```
[Seed URLs]
     |
     ▼
[URL Frontier / Priority Queue]
```



Bloom Filter for deduplication: A probabilistic data structure that can test set membership. Adding a URL to the filter never produces false negatives (if it says "not seen," it hasn't been seen). Small false positive rate is acceptable (occasionally skipping a valid URL). A Bloom filter for 10 billion URLs uses ~1.2 GB of memory (vs 200 GB for a hash set).

Politeness: Parse and cache `robots.txt` per domain. Enforce minimum crawl delay (e.g., 1 request per domain per second). Maintain a per-domain crawl schedule.

URL Frontier: Priority queue that orders URLs by importance score (PageRank, recency, freshness). Separate queues per domain for politeness enforcement.

9.3 Design a Key-Value Store (Dynamo/Cassandra Style)

Requirements

- GET(key) → value, PUT(key, value), DELETE(key)
- Horizontally scalable, eventually consistent
- Highly available (AP system)
- Handles node failures gracefully

Core Architecture Decisions

Consistent Hashing: Keys are distributed across nodes using consistent hashing. Adding or removing nodes moves only K/N keys (vs naive modulo hash which moves nearly all keys).

```
Consistent Hash Ring:
      Node A
     /      \
Node D       Node B
     \      /
      Node C

Key X → hash(X) → find nearest node clockwise on ring
```

Virtual Nodes (vnodes): Each physical node is represented by multiple points (vnodes) on the ring. This ensures more even distribution and reduces hotspots when nodes are added/removed.

Replication: Each key is replicated to N nodes (typically N=3) — the coordinator node plus the next N-1 nodes clockwise on the ring.

Quorum Reads/Writes (R, W, N):

- N = total replicas (3)
- W = write quorum (minimum replicas that must acknowledge write)
- R = read quorum (minimum replicas that must respond to read)
- Condition for strong consistency: R + W > N (e.g., R=2, W=2, N=3)
- Tunable: W=1, R=1 → high availability, eventual consistency; W=3, R=3 → strong consistency, lower availability

Hinted Handoff: If a replica node is temporarily unavailable, the coordinator stores a "hint" on another node. When the unavailable node comes back, the hint is replayed to bring it up to date.

Read Repair: When a coordinator reads from multiple replicas and detects inconsistency, it proactively repairs the stale replica with the up-to-date value.

Vector Clocks: Used to track causality and detect conflicts. Each value is tagged with a vector clock showing which version each node has seen. If two values have conflicting vector clocks (concurrent writes), the application must resolve the conflict (e.g., shopping cart — merge; last-write-wins for simple values).

9.4 Design a News Feed System (Twitter/Facebook)

Requirements

- Users post tweets/posts
- Users follow other users

- Timeline shows posts from followed users in reverse chronological order
- 300M MAU, 1M posts/day

Back-of-Envelope

```
DAU: 150M
Posts/day: 1M → ~12 writes/sec
Timeline reads: 150M users × 5 reads/day = 750M reads/day → ~8,700 reads/sec
Peak reads: ~50,000 reads/sec
Read:Write ratio ≈ 4,000:1 → extremely read-heavy
```

Fan-out on Write vs Fan-out on Read

Fan-out on Write (Push model): When a user posts, immediately write the post to the timeline cache (Redis Sorted Set) of all followers.

```
User A (1000 followers) posts:
→ For each of 1000 followers:
  → Insert into follower's Redis timeline list
Reads are instant (pre-computed). But: celebrity with 100M followers = 100M
writes per post!
```

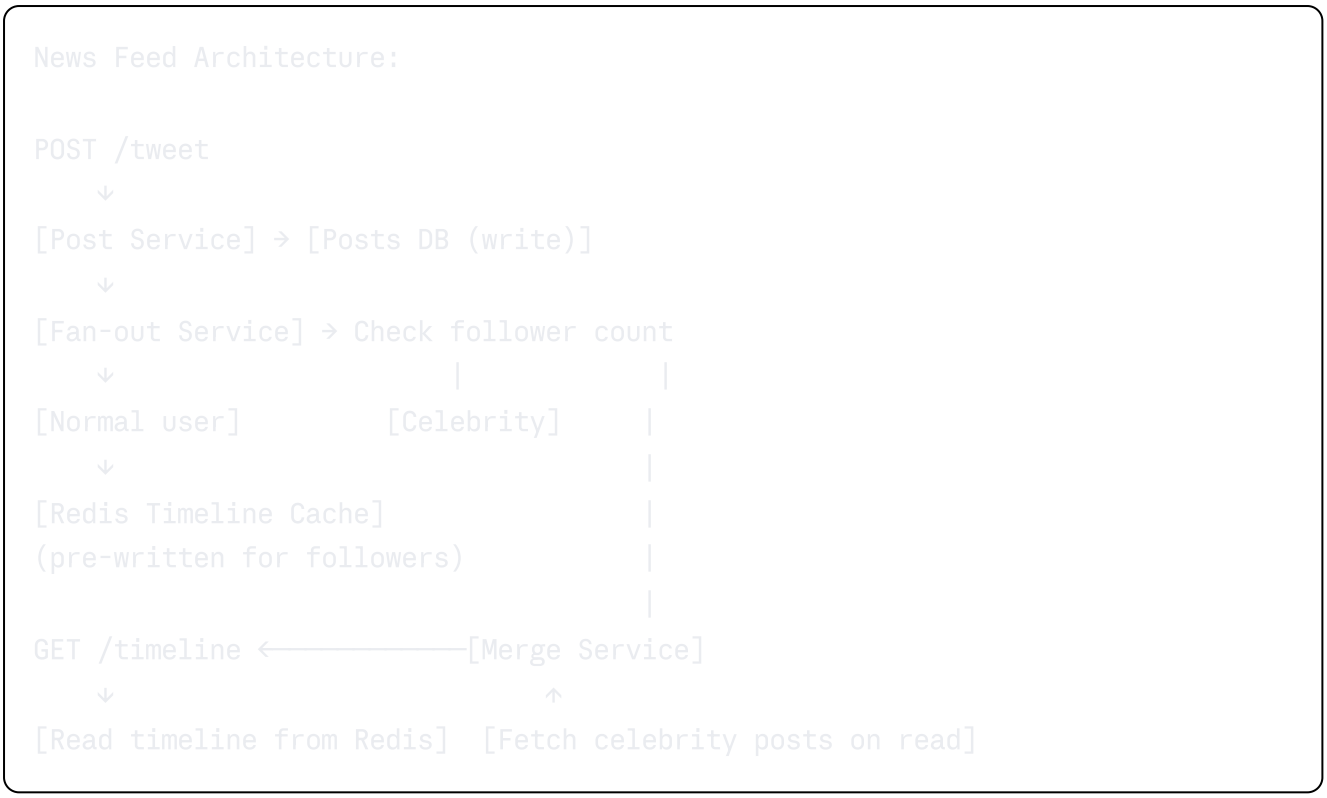
Fan-out on Read (Pull model): When a user requests their timeline, fetch posts from all followed users and merge.

```
User X requests timeline:
→ Fetch X's following list (200 users)
→ For each: fetch last 10 posts from their post DB
→ Merge and sort 2000 posts
→ Return top 20
```

Fast writes but slow, expensive reads.

Hybrid Approach (Twitter's actual approach):

- Regular users (< 1M followers): Fan-out on write (push to Redis timeline)
- Celebrities/influencers (> 1M followers): Fan-out on read (fetch on demand)
- On read: merge pre-computed timeline + real-time fetch from followed celebrities



Data Schema

sql

```sql
-- Post table (sharded by user_id)
CREATE TABLE posts (
    post_id     BIGINT PRIMARY KEY,  -- Snowflake ID (time-sortable)
    user_id     BIGINT NOT NULL,
    content     TEXT,
    media_urls  JSON,
    created_at  TIMESTAMP DEFAULT NOW(),
    INDEX idx_user_created (user_id, created_at DESC)
);

-- Follower graph
CREATE TABLE follows (
    follower_id BIGINT,
    followee_id BIGINT,
    created_at  TIMESTAMP,
    PRIMARY KEY (follower_id, followee_id)
);
```

9.5 Design a Chat/Messaging System (WhatsApp)

Requirements

- One-on-one and group messaging
- Message delivery receipts (sent, delivered, read)
- Online presence/status
- End-to-end encryption consideration
- 2 billion users, 100B messages/day

Back-of-Envelope

```
100B messages/day ÷ 86,400 sec ≈ 1.16M messages/sec
Average message: 100 bytes
Bandwidth: 1.16M × 100 bytes = 116 MB/s write bandwidth
Storage: 100B × 100 bytes = 10 TB/day (before replication)
```

WebSocket for Real-Time Communication

HTTP is request-response — the server can't push to the client. **WebSockets** provide a persistent, bidirectional connection ideal for chat.

```
WebSocket Connection Lifecycle:
Client → HTTP Upgrade Request → [Chat Server]
Client ←── 101 Switching Protocols ─── [Chat Server]
Client ←═══════════ WebSocket (bidirectional) ═══════════→ [Chat Server]
```

Message Flow

```
Alice → [Chat Server A] → [Message Queue (Kafka)] → [Chat Server B] → Bob
                                  |
                            [Message DB]
                            (Cassandra)
                                  |
                            (if Bob offline)
                                  |
                        [Push Notification Service]
                            (APNs, FCM)
```

Why Cassandra for messages? Messages are time-series data — append-only, queried by conversation ID and sorted by timestamp. Cassandra's partition key

(conversation_id) + clustering key (timestamp DESC) is a perfect fit. Scales to millions of writes/sec.

Online Presence System

```
User connects → POST heartbeat to Presence Service every 5 seconds
User disconnects → After 10s without heartbeat, mark offline
Other users query presence: GET /presence/{user_id} → check Redis (TTL=10s)

Redis: SET presence:user_123 "online" EX 10
On heartbeat: SET presence:user_123 "online" EX 10 (refresh TTL)
After 10s without heartbeat: key expires → user is offline
```

Message Delivery Receipts

- **Sent (✓):** Message written to server's DB
- **Delivered (✓ ✓):** Server received ACK from recipient's device
- **Read (✓ ✓ blue):** Recipient's app sent read receipt back to server

---

9.6 Design a Video Streaming Platform (YouTube)

Requirements

- Upload videos; store in multiple resolutions
- Stream videos to millions of concurrent viewers
- Support adaptive bitrate streaming
- 500 hours of video uploaded/minute; 1B views/day

Back-of-Envelope

```
Upload: 500 hours/min × 60 min/hr = 30,000 min of video/min ≈ 500 MB/min
After transcoding (3 formats): 1.5 GB/min = 25 MB/s storage writes
View bandwidth: 1B views/day × avg 10 min × 4 Mbps = 4.63 Pbps (CDN needed!)
```

Upload Pipeline

```
User → [Upload API] → [S3 Raw Storage]
                ↓
        [Kafka: upload event]
                ↓
        [Transcoding Workers]
```

```
        (FFmpeg: 1080p, 720p, 480p, 360p)
                ↓
        [S3 Transcoded Storage]
                ↓
        [CDN Push / Invalidate]
                ↓
        [Database: video metadata]
```

Adaptive Bitrate Streaming (ABR): Video is split into 5-10 second segments at multiple quality levels. The client player monitors network conditions and requests the highest quality segment it can receive without buffering.

HLS (HTTP Live Streaming) / DASH: Standard protocols for ABR. A manifest file (.m3u8 for HLS) lists all available segment files and quality levels. The player downloads the manifest, then selects and downloads segments.

```
Manifest file (HLS):
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=1280x720
720p/segment.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=400000,RESOLUTION=640x360
360p/segment.m3u8
```

9.7 Design a Ride-Sharing Service (Uber)

Requirements

- Riders request rides; drivers receive nearby requests
- Real-time location tracking
- Matching algorithm (nearest available driver)
- Surge pricing based on supply/demand
- ETA calculation

Back-of-Envelope

```
Drivers send location every 5 seconds
10M active drivers × (1 update / 5 sec) = 2M location updates/sec
Each update: driver_id (8B) + lat (8B) + lng (8B) + timestamp (8B) = 32 bytes
Bandwidth: 2M × 32 bytes = 64 MB/s location writes
```

Architecture

Location Tracking:

```
Driver App → WebSocket/HTTP → [Location Service]
                                      ↓
                              [Redis Geospatial Index]
                              GEOADD drivers_online lng lat driver_id
                              GEORADIUS drivers_online lat lng 5 km
```

Redis `GEORADD` and `GEORADIUS` commands enable efficient geospatial queries. Redis stores locations as a sorted set with geohash scores.

Matching Algorithm:

```
Rider requests ride at location (lat, lng)
[Matching Service]:
1. GEORADIUS "drivers:available" lat lng 5 km ASC COUNT 10
2. Filter by driver preferences (car type, rating)
3. Rank by ETA (pre-computed via routing service like OSRM/Google Maps)
4. Offer trip to nearest driver
5. If driver rejects, offer to next driver
```

Geohashing: Divide the map into a grid of cells represented by alphanumeric strings. Proximity = shared prefix. "9q8yy" and "9q8yz" are adjacent cells. Allows fast proximity queries in any database.

9.8 Design a Distributed Unique ID Generator

Requirements

- Generate globally unique IDs across multiple servers
- IDs should be sortable by time (for efficient DB indexing)
- High throughput: 10,000+ IDs/sec per server
- Available and fault-tolerant

Approaches

Database Auto-Increment: Simple but single point of failure. Multi-master approach with different start values and step sizes (Server 1: 1,3,5...; Server 2: 2,4,6...) doesn't scale and is complex.

UUID (Universally Unique Identifier): 128-bit random value; virtually zero collision probability. Version 4 (random) is not time-sortable. Version 7 (time-based, 2023 RFC) is time-sortable. Stored as 16 bytes (efficient) or 36-char string (wasteful).

Twitter Snowflake ID: 64-bit integer, globally unique, time-sortable:

```
|--- 41 bits ---|-- 10 bits --|-- 12 bits --|
  Timestamp (ms)   Machine ID   Sequence #
  (epoch: custom)  (datacenter  (resets per ms)
                    + worker)
```

- **41-bit timestamp:** ms since custom epoch → ~69 years of IDs
- **10-bit machine ID:** 5-bit datacenter + 5-bit worker → 32 DCs × 32 workers = 1024 servers
- **12-bit sequence:** 4096 IDs per millisecond per server → 4M IDs/sec per server

java

```java
// Snowflake ID structure
long timestamp = currentTimeMillis() - EPOCH;
long id = (timestamp << 22) | (datacenterId << 17) | (workerId << 12) | sequence;
```

Clock skew problem: If server clock goes backward, IDs may be non-unique. Solution: refuse to generate IDs until clock catches up; use NTP for time synchronization.

9.9 Design a Notification System

Requirements

- Deliver notifications via push (iOS APNs, Android FCM), SMS, email
- Prioritize critical notifications (OTP, payment alerts) over marketing
- Handle millions of notifications/day
- Track delivery status

Architecture

```
[Notification API] ← Trigger from any service
        ↓
[Kafka Topic: notifications] (buffering, replay)
        ↓
[Notification Service]
```

```
  - Validate & enrich (fetch user preferences, tokens)
  - Route by channel (push/SMS/email)
  - Deduplicate (don't send same notification twice)
        ↓                ↓                ↓
[APNs/FCM Worker]  [SMS Worker]  [Email Worker]
(AWS SNS Mobile)   (Twilio)      (SendGrid/SES)
        ↓                ↓                ↓
[Delivery DB] (track: pending → sent → delivered → failed)
```

Priority queues: Use separate Kafka topics for high-priority (OTP, alerts) and low-priority (marketing) notifications. High-priority consumers get more workers.

Rate limiting: Respect per-user notification limits (e.g., max 10 push notifications/hour) to avoid user disabling notifications.

9.10 Design a Consistent Hashing System

Consistent hashing solves the problem of redistributing data when nodes are added or removed from a distributed system. In naive modular hashing (`hash(key) % N`), changing N causes ~100% of keys to remap to different nodes.

```
Naive hash: N=3 → N=4
key "foo": hash("foo") % 3 = 2 → Node C
key "foo": hash("foo") % 4 = 1 → Node B (different! must move data)
Nearly all keys move.

Consistent hashing: N=3 → N=4
Only keys that "belonged" to the node adjacent to the new node need to move.
On average, only K/N keys are remapped (K = total keys, N = number of nodes).
```

Ring Architecture

```
Hash ring (0 to 2^32 - 1):


          0
       ↗     ↖

Node A          Node B
  ↑                ↓
Node D          Node C
       ↘     ↙
         2^32-1
```

```
Key K → hash(K) → find first node clockwise on ring
```

Virtual Nodes

Without virtual nodes, if Node A is responsible for a large arc of the ring, it stores more data than Node B (small arc). Virtual nodes create multiple ring positions per physical node:

```
Each physical node has V virtual nodes:
Node A: positions at 10, 180, 340
Node B: positions at 60, 200, 320
Node C: positions at 120, 260, 380

More evenly distributed. When Node A is removed, its keys are distributed to
multiple remaining nodes, not just one.
```

Use cases for consistent hashing: Load balancing (distribute requests to servers), distributed caching (Redis Cluster, Memcached), database sharding (Cassandra, DynamoDB partition placement).

9.11 Design a Typeahead/Autocomplete

Requirements

- As user types, show top-5 completions in < 100ms
- Suggestions ranked by search frequency
- Personalized suggestions
- 10M DAU, average 10 characters typed per search

Core Data Structure: Trie

A **Trie** (prefix tree) stores strings as paths, enabling O(L) prefix lookup where L is the query length.

```
Trie for ["apple", "app", "application", "apply"]:

(root)
  └─ a
     └─ p
        └─ p (word: "app")
           ├─ l
           |  └─ e (word: "apple")
           |  └─ i → c → a → t → i → o → n (word: "application")
           |  └─ y (word: "apply")
```

For autocomplete: Store top-N suggestions at each trie node (top suggestions within this subtree). Avoids DFS traversal — simply read the pre-stored suggestions.

Scaling the Trie:

- **Distributed trie:** Shard the trie by first character or prefix
- **Cache in Redis:** Store `prefix → [suggestion1, suggestion2, suggestion3]` as Redis sorted sets (scored by frequency)
- **CDN caching:** Popular short prefixes (2-3 chars) can be CDN-cached

```
Redis autocomplete structure:
ZADD autocomplete:se 1000 "search"
ZADD autocomplete:se 800 "settings"
ZADD autocomplete:se 600 "security"
ZREVRANGE autocomplete:se 0 4  → top 5 by score
```

Updating trie: Don't update in real-time (too expensive). Collect search queries in Kafka, aggregate hourly/daily, recompute and reload trie from batch job.

9.12 Design a Payment System

Requirements

- Process payments (credit card, digital wallet)
- Prevent double-charging
- Idempotent operations
- Financial audit trail
- PCI-DSS compliance

```
User → [Payment API]
          ↓
   [Idempotency Check] → Redis (idempotency key store)
          ↓
   [Payment Processor] → [Stripe/PayPal/Bank API]
          ↓
   [Ledger Service] → double-entry bookkeeping
          ↓
   [Event: PaymentCompleted] → Kafka
          ↓
   [Order Service, Notification Service, Analytics]
```

Double-entry bookkeeping: Every financial transaction records two entries: one debit and one credit. Balance always sums to zero, making the system self-auditing.

```
Payment of $100 from User → Merchant:
DEBIT  User account:     -$100
CREDIT Merchant account: +$100
(net = $0, no money created or destroyed)
```

Idempotency in payments: The client generates a UUID idempotency key. The payment API stores the key and result. Retried requests with the same key return the stored result without re-charging.

Reconciliation: Periodically compare your internal ledger against the payment processor's records (Stripe webhook events vs your DB). Detect and flag discrepancies.

PCI-DSS: Payment Card Industry Data Security Standard. Never store raw card numbers — store only last 4 digits + tokenized representation. Use a PCI-compliant provider (Stripe handles PCI compliance for you when using client-side tokenization).

9.13 Design a Distributed Cache (Redis Cluster)

Requirements

- High-throughput caching
- Fault tolerance (cache server failures)
- Horizontal scaling
- Sub-millisecond reads

Redis Cluster Architecture

```
Redis Cluster (6 nodes: 3 masters + 3 replicas):

[Master 1] ←replicates→ [Replica 1]  (slots 0-5460)
[Master 2] ←replicates→ [Replica 2]  (slots 5461-10922)
[Master 3] ←replicates→ [Replica 3]  (slots 10923-16383)

Client sends GET "user:123":
hash_slot = CRC16("user:123") % 16384 = 7532
→ Route to Master 2 (handles slots 5461-10922)
```

Redis Cluster uses **hash slots** (not consistent hashing directly). 16,384 hash slots are divided among master nodes. Each key maps to a slot via CRC16. Clients use the cluster topology map to route directly to the correct node.

Cache eviction policies: When Redis memory is full:

- `allkeys-lru`: Evict least recently used keys from all keys (recommended for caches)
- `volatile-lru`: Evict LRU from keys with TTL set
- `allkeys-random`: Evict random key
- `noeviction`: Return error on writes (for persistent data stores)

Chapter 9 — Quick Revision Box

> - **URL Shortener:** Base62 encoding of auto-increment ID; cache hot redirects in Redis; use 302 for analytics
> - **Web Crawler:** Bloom filter for URL deduplication; politeness via robots.txt + per-domain crawl delay
> - **Distributed KV Store:** Consistent hashing + virtual nodes; quorum reads/writes (R+W>N); hinted handoff for temporary failures
> - **News Feed:** Fan-out on write for regular users; fan-out on read for celebrities; merge on read
> - **Chat System:** WebSocket for bidirectional real-time; Cassandra (partition=conversation, cluster=timestamp); Redis for presence
> - **Video Streaming:** Transcoding pipeline → multiple resolutions; HLS/DASH for adaptive bitrate; CDN for global delivery
> - **Payments:** Idempotency keys for double-charge prevention; double-entry ledger for audit trail; never store raw card numbers

---

Chapter 10: Low-Level Design (LLD) Fundamentals {#chapter-10}

10.1 What is LLD?

Low-Level Design (LLD) focuses on the detailed design of individual components, classes, and modules that make up a system. While HLD answers "what components exist and how do they interact?", LLD answers "how is each component implemented?"

LLD interviews typically ask you to design a class structure for a system: parking lot, elevator, library, chess game. The evaluator assesses your ability to apply SOLID principles, choose appropriate design patterns, design clean class hierarchies, and define methods and responsibilities clearly.

10.2 SOLID Principles

Single Responsibility Principle (SRP): A class should have one — and only one — reason to change. Each class is responsible for a single piece of functionality.

```java
// VIOLATION: Invoice class does too much
class Invoice {
    void calculateTotal() { ... }
    void printInvoice() { ... }    // ← different responsibility
    void saveToDatabase() { ... }  // ← different responsibility
}

// CORRECT: Separate classes for separate concerns
class Invoice { void calculateTotal() { ... } }
class InvoicePrinter { void print(Invoice inv) { ... } }
class InvoiceRepository { void save(Invoice inv) { ... } }
```

Open/Closed Principle (OCP): Software entities should be open for extension but closed for modification. Add new behavior without changing existing code.

```java
```

```
// Instead of: if (shape == "circle") ... else if (shape == "rectangle")
// Use polymorphism:
interface Shape { double area(); }
class Circle implements Shape { double area() { return Math.PI * r * r; } }
class Rectangle implements Shape { double area() { return width * height; } }
// Add new shape: just implement Shape, no modification to existing code
```

Dependency Inversion Principle (DIP): High-level modules should not depend on low-level modules. Both should depend on abstractions.

java

```
// VIOLATION: High-level depends on low-level
class OrderService {
    MySQLDatabase db = new MySQLDatabase(); // ← concrete dependency
}

// CORRECT: Depend on abstraction
interface Database { void save(Order o); }
class OrderService {
    private Database db;
    OrderService(Database db) { this.db = db; } // ← dependency injection
}
```

10.3 Design Patterns

Design patterns are reusable solutions to commonly occurring problems in software design. The 23 Gang of Four (GoF) patterns are divided into three categories.

Creational Patterns

Singleton Pattern

Intent: Ensure only one instance of a class exists; provide a global access point.

```
Singleton UML:
+-------------------+
|    Singleton      |
|-------------------|
| -instance: Singleton (static) |
|-------------------|
| +getInstance(): Singleton (static) |
| +operation()      |
+-------------------+
```

java

```java
// Thread-safe Singleton using Double-Checked Locking
public class DatabaseConnectionPool {
    private static volatile DatabaseConnectionPool instance;
    private final List<Connection> connections;

    private DatabaseConnectionPool() {
        connections = initializePool(10);
    }

    public static DatabaseConnectionPool getInstance() {
        if (instance == null) {                    // 1st check (no locking)
            synchronized (DatabaseConnectionPool.class) {
                if (instance == null) {            // 2nd check (with lock)
                    instance = new DatabaseConnectionPool();
                }
            }
        }
        return instance;
    }
}
```

Real-world example: Database connection pool, logger, configuration manager.

Factory Method Pattern

Intent: Define an interface for creating an object, but let subclasses decide which class to instantiate.

java

```java
// Product interface
interface Notification {
    void send(String message);
}

// Concrete products
class SMSNotification implements Notification {
    public void send(String message) { /* SMS logic */ }
}
class EmailNotification implements Notification {
    public void send(String message) { /* Email logic */ }
}

// Factory method
class NotificationFactory {
    public static Notification create(String type) {
        return switch (type) {
            case "SMS" -> new SMSNotification();
            case "EMAIL" -> new EmailNotification();
            default -> throw new IllegalArgumentException("Unknown type: " + type
        };
    }
}
```

Real-world example: Logger factories, database driver factories, UI toolkit factories.

Builder Pattern

Intent: Construct complex objects step-by-step; separate construction from representation.

java

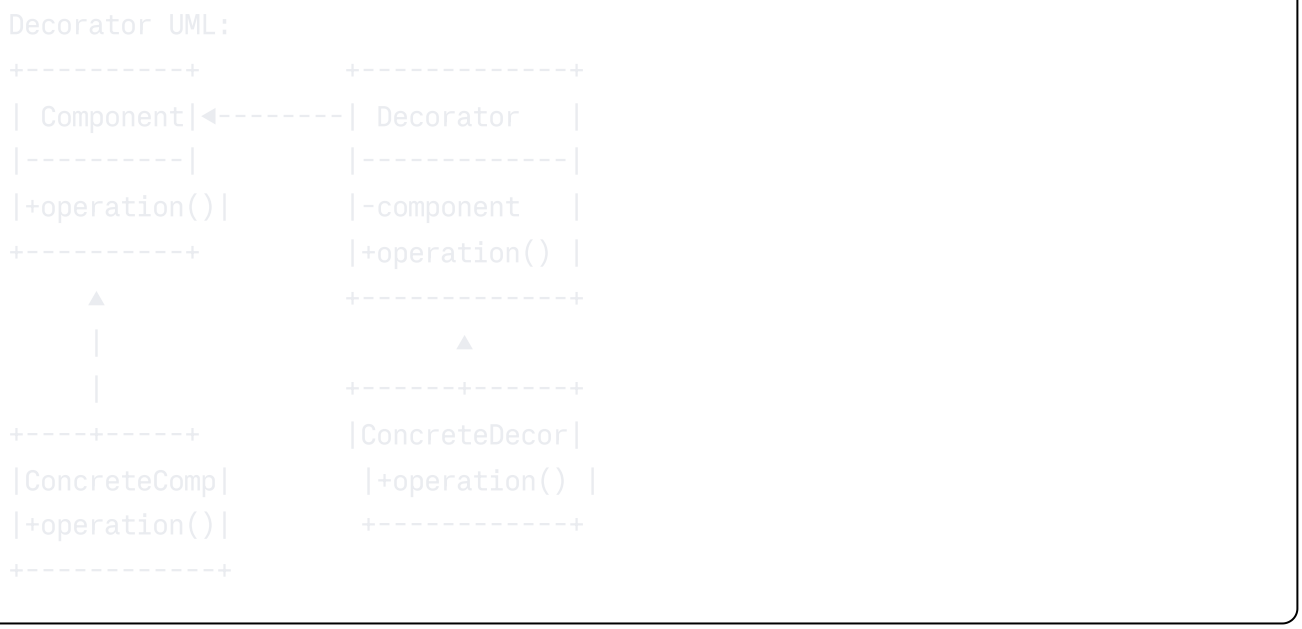
```java
// Without Builder: telescoping constructors (anti-pattern)
new Pizza(size, crust, sauce, cheese, topping1, topping2); // hard to read!

// With Builder:
Pizza pizza = new Pizza.Builder("large")
    .crust("thin")
    .sauce("tomato")
    .cheese("mozzarella")
    .topping("mushrooms")
    .build();
```

Real-world example: `StringBuilder`, `HttpRequest.Builder` (Java 11), query builders, configuration builders.

Structural Patterns

Decorator Pattern

Intent: Attach additional responsibilities to an object dynamically, without modifying its class. Decorators provide a flexible alternative to subclassing.



java

```java
interface Coffee { double getCost(); String getDescription(); }

class SimpleCoffee implements Coffee {
    public double getCost() { return 1.0; }
    public String getDescription() { return "Coffee"; }
}

class MilkDecorator implements Coffee {
    private Coffee coffee;
    MilkDecorator(Coffee coffee) { this.coffee = coffee; }
    public double getCost() { return coffee.getCost() + 0.5; }
    public String getDescription() { return coffee.getDescription() + ", Milk"; }
}

class SugarDecorator implements Coffee {
    private Coffee coffee;
    SugarDecorator(Coffee coffee) { this.coffee = coffee; }
    public double getCost() { return coffee.getCost() + 0.25; }
    public String getDescription() { return coffee.getDescription() + ", Sugar"; }
}

// Usage:
Coffee c = new SugarDecorator(new MilkDecorator(new SimpleCoffee()));
c.getDescription(); // → "Coffee, Milk, Sugar"
c.getCost();        // → 1.75
```

Real-world example: Java I/O streams (`BufferedReader` wraps `FileReader`), web middleware, compression/encryption wrappers.

Proxy Pattern

Intent: Provide a surrogate or placeholder for another object to control access to it.

Types: Virtual proxy (lazy initialization of expensive objects), Protection proxy (access control), Remote proxy (RPC stub), Caching proxy (cache results).

```java
```

```java
interface Image { void display(); }

class RealImage implements Image {
    private String filename;
    RealImage(String filename) {
        this.filename = filename;
        loadFromDisk(); // expensive
    }
    private void loadFromDisk() { System.out.println("Loading " + filename); }
    public void display() { System.out.println("Displaying " + filename); }
}

class ImageProxy implements Image {  // Virtual proxy - lazy loading
    private RealImage realImage;
    private String filename;

    ImageProxy(String filename) { this.filename = filename; } // no load yet!

    public void display() {
        if (realImage == null) {
            realImage = new RealImage(filename); // load only when needed
        }
        realImage.display();
    }
}
```

Adapter Pattern

Intent: Convert the interface of a class into another interface that clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.

java

```java
// Existing class with incompatible interface
class OldPaymentSystem {
    void makePayment(double amount, String cardNumber) { ... }
}

// Target interface we want to use
interface PaymentProcessor {
    void processPayment(PaymentRequest request);
}

// Adapter
class PaymentAdapter implements PaymentProcessor {
    private OldPaymentSystem oldSystem;
    PaymentAdapter(OldPaymentSystem old) { this.oldSystem = old; }

    public void processPayment(PaymentRequest request) {
        oldSystem.makePayment(request.getAmount(), request.getCardNumber());
    }
}
```

Facade Pattern

Intent: Provide a simplified interface to a complex subsystem.

java

```java
// Complex subsystem
class VideoConverter { ... }
class AudioCodec { ... }
class MetadataWriter { ... }

// Facade - simple interface
class VideoProcessingFacade {
    private VideoConverter converter = new VideoConverter();
    private AudioCodec codec = new AudioCodec();
    private MetadataWriter writer = new MetadataWriter();

    public void processVideo(String inputFile, String outputFormat) {
        converter.convert(inputFile, outputFormat);
        codec.encode(inputFile);
        writer.writeMetadata(inputFile);
    }
}
```

Behavioral Patterns

Observer Pattern

Intent: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically.

```java
interface Observer { void update(String event, Object data); }
interface Subject {
    void subscribe(Observer o);
    void unsubscribe(Observer o);
    void notifyObservers(String event, Object data);
}

class EventBus implements Subject {
    private Map<String, List<Observer>> listeners = new HashMap<>();

    public void subscribe(String event, Observer o) {
        listeners.computeIfAbsent(event, k -> new ArrayList<>()).add(o);
    }

    public void publish(String event, Object data) {
        listeners.getOrDefault(event, List.of()).forEach(o -> o.update(event, da
    }
}

// Usage:
EventBus bus = new EventBus();
bus.subscribe("ORDER_PLACED", (event, data) -> sendEmail((Order)data));
bus.subscribe("ORDER_PLACED", (event, data) -> updateInventory((Order)data));
bus.publish("ORDER_PLACED", order); // notifies all subscribers
```

Real-world example: Event listeners (UI frameworks), Kafka consumers, MVC model-view binding, reactive streams.

Strategy Pattern

Intent: Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

java

```java
interface SortStrategy { void sort(int[] arr); }

class QuickSort implements SortStrategy { public void sort(int[] arr) { /* quicks
class MergeSort implements SortStrategy { public void sort(int[] arr) { /* merges
class BubbleSort implements SortStrategy { public void sort(int[] arr) { /* bubbl

class DataProcessor {
    private SortStrategy strategy;
    DataProcessor(SortStrategy strategy) { this.strategy = strategy; }
    void setStrategy(SortStrategy strategy) { this.strategy = strategy; }
    void process(int[] data) { strategy.sort(data); }
}

// Usage: switch strategy at runtime
DataProcessor dp = new DataProcessor(new QuickSort());
dp.process(largeArray);
dp.setStrategy(new MergeSort()); // switch strategy without changing DataProcesso
```

Real-world example: Payment strategy (credit card, PayPal, crypto), compression strategy, routing strategy.

Command Pattern

Intent: Encapsulate a request as an object, allowing parameterization, queueing, logging, and undoable operations.

java

```
interface Command { void execute(); void undo(); }

class LightOnCommand implements Command {
    private Light light;
    LightOnCommand(Light l) { this.light = l; }
    public void execute() { light.turnOn(); }
    public void undo() { light.turnOff(); }
}

class RemoteControl {
    private Stack<Command> history = new Stack<>();
    public void pressButton(Command cmd) {
        cmd.execute();
        history.push(cmd);
    }
    public void pressUndo() {
        if (!history.isEmpty()) history.pop().undo();
    }
}
```

Real-world example: Text editor undo/redo, GUI button actions, database transactions, job queues.

Template Method Pattern

Intent: Define the skeleton of an algorithm in a method, deferring some steps to subclasses. Lets subclasses redefine specific steps without changing the algorithm's structure.

java

```
abstract class DataImporter {
    // Template method
    public final void importData(String source) {
        String raw = readData(source);      // step 1 - abstract
        String processed = processData(raw); // step 2 - abstract
        saveData(processed);                 // step 3 - concrete
    }

    protected abstract String readData(String source);
    protected abstract String processData(String data);

    private void saveData(String data) {
        database.save(data); // shared logic
    }
}

class CSVImporter extends DataImporter {
    protected String readData(String s) { /* parse CSV */ return ...; }
    protected String processData(String d) { /* transform */ return ...; }
}
```

Complete Design Pattern Reference

Category	Pattern	Intent	Key Participants
Creational	Singleton	One instance, global access	Singleton class
Creational	Factory Method	Subclasses create objects	Creator, Product
Creational	Abstract Factory	Create families of objects	AbstractFactory, ConcreteFactory
Creational	Builder	Construct complex objects step-by-step	Builder, Director
Creational	Prototype	Clone existing objects	Prototype, ConcretePrototype
Structural	Adapter	Convert interfaces	Adapter, Adaptee, Target
Structural	Decorator	Add responsibilities dynamically	Component, Decorator

Category	Pattern	Intent	Key Participants
Structural	Facade	Simplified interface to subsystem	Facade, Subsystem
Structural	Proxy	Control access to object	Proxy, RealSubject
Structural	Composite	Tree structures of objects	Component, Leaf, Composite
Structural	Bridge	Decouple abstraction from implementation	Abstraction, Implementor
Structural	Flyweight	Share state to support many fine-grained objects	Flyweight, FlyweightFactory
Behavioral	Observer	Notify dependents of state changes	Subject, Observer
Behavioral	Strategy	Interchangeable algorithms	Strategy, Context
Behavioral	Command	Encapsulate requests as objects	Command, Invoker, Receiver
Behavioral	Iterator	Sequential access to elements	Iterator, Aggregate
Behavioral	Template Method	Algorithm skeleton with subclass steps	AbstractClass, ConcreteClass
Behavioral	State	Object changes behavior when state changes	State, Context
Behavioral	Chain of Responsibility	Pass requests along a chain	Handler, ConcreteHandler
Behavioral	Mediator	Centralize communication between objects	Mediator, Colleague
Behavioral	Memento	Capture and restore object state	Memento, Originator, Caretaker
Behavioral	Visitor	Add operations to objects without changing them	Visitor, Element

10.4 Object-Oriented Design: Parking Lot

```
Requirements:
- Multiple floors, multiple spots per floor
- Spot types: motorcycle, compact, large
- Entry/exit with tickets
- Calculate parking fees
- Real-time availability

Class Design:

+----------------+       +------------------+
| ParkingLot     |<>---->| ParkingFloor     |
|----------------|       |------------------|
| -floors: List  |       | -spots: List     |
| +getAvailable()|       | +findAvailable() |
| +parkVehicle() |       +------------------+
| +processExit() |                <>
+----------------+                 |
        |                 +--------+-------+
        |                 | ParkingSpot    |
        |                 |----------------|
        |                 | -spotId: int   |
   +----+----+            | -type: SpotType|
   | Ticket  |            | -isOccupied    |
   |---------|            | +park(vehicle) |
   | -id     |            | +unpark()      |
   | -entry  |            +----------------+
   | -vehicle|
   | +getFee()|    +-----------+   +-----------+   +-----------+
   +---------+    | Vehicle   |   |Motorcycle|   | Car       |
                  |-----------|   |-----------|   |-----------|
                  | -plate    |   | type=MOTO|   | type=COMP|
                  | +getType()|   +-----------+   +-----------+
                  +-----------+
```

10.5 OOD: LRU Cache Design

Design an LRU cache with O(1) get and O(1) put operations.

Key insight: Use a **HashMap + Doubly Linked List**. HashMap provides O(1) key lookup; Doubly Linked List maintains access order (recently used = head, least recently used = tail).

java

```java
class LRUCache {
    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0); // dummy head
    private final Node tail = new Node(0, 0); // dummy tail

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key);
        moveToHead(node); // mark as recently used
        return node.val;
    }

    public void put(int key, int val) {
        if (map.containsKey(key)) {
            Node node = map.get(key);
            node.val = val;
            moveToHead(node);
        } else {
            if (map.size() == capacity) {
                Node lru = removeTail(); // evict LRU
                map.remove(lru.key);
            }
            Node newNode = new Node(key, val);
            addToHead(newNode);
            map.put(key, newNode);
        }
    }

    private void addToHead(Node n) { n.next = head.next; n.prev = head; head.nex
    private void removeNode(Node n) { n.prev.next = n.next; n.next.prev = n.prev
    private void moveToHead(Node n) { removeNode(n); addToHead(n); }
    private Node removeTail() { Node n = tail.prev; removeNode(n); return n; }

    static class Node { int key, val; Node prev, next; Node(int k, int v) { key=
}
```

Chapter 10 — Quick Revision Box

> - **SRP:** One reason to change; split fat classes; **OCP:** extend without modifying; **DIP:** depend on abstractions, not concretions
> - **Singleton:** volatile + double-checked locking for thread safety; watch for Singleton abuse
> - **Factory Method:** subclasses decide which object to create; **Builder:** step-by-step complex object construction
> - **Observer:** one-to-many event notifications; Subject notifies all registered Observers
> - **Strategy:** swap algorithms at runtime; eliminates switch/if-else on type
> - **Decorator:** add behavior by wrapping objects; used in Java I/O streams
> - **LRU Cache:** HashMap (O(1) lookup) + Doubly Linked List (O(1) eviction); moveToHead on access

---

Chapter 11: Advanced LLD {#chapter-11}

11.1 Concurrency in LLD

Concurrent programming introduces race conditions, deadlocks, and visibility issues. Understanding concurrency patterns is essential for LLD interviews at senior levels.

Thread-Safe Singleton (revisited)

The `volatile` keyword in Java ensures the `instance` variable is not cached in CPU registers — all threads see the latest written value from main memory. Without `volatile`, the double-checked locking pattern is broken.

Producer-Consumer Pattern with Blocking Queue

```java
```

```
class ProducerConsumer {
    private final BlockingQueue<Task> queue;
    private final ExecutorService consumers;

    public ProducerConsumer(int capacity, int numConsumers) {
        queue = new LinkedBlockingQueue<>(capacity);
        consumers = Executors.newFixedThreadPool(numConsumers);
        for (int i = 0; i < numConsumers; i++) {
            consumers.submit(this::consumeTasks);
        }
    }

    public void produce(Task task) throws InterruptedException {
        queue.put(task); // blocks if queue is full (backpressure)
    }

    private void consumeTasks() {
        while (!Thread.currentThread().isInterrupted()) {
            try {
                Task task = queue.take(); // blocks if queue is empty
                processTask(task);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```

Read-Write Lock Pattern

When reads are frequent and writes are rare, a `ReadWriteLock` allows multiple concurrent readers but exclusive access for writers.

java

```
class Cache {
    private final ReadWriteLock lock = new ReentrantReadWriteLock();
    private final Map<String, Object> data = new HashMap<>();

    public Object read(String key) {
        lock.readLock().lock();
        try { return data.get(key); }
        finally { lock.readLock().unlock(); }
    }

    public void write(String key, Object value) {
        lock.writeLock().lock();
        try { data.put(key, value); }
        finally { lock.writeLock().unlock(); }
    }
}
```

11.2 Domain-Driven Design (DDD)

Domain-Driven Design is an approach to software development that aligns the software model with the business domain. Key concepts:

Entity: An object defined by its identity (ID), not its attributes. Two users with different IDs are different entities even if they have the same name.

Value Object: An object defined by its attributes, with no identity. `Money(100, USD)` == `Money(100, USD)`. Value objects are immutable.

Aggregate: A cluster of domain objects treated as a single unit. One entity is the **Aggregate Root** — the only entry point from outside. An `Order` aggregate contains `OrderItems` — you never modify an `OrderItem` directly, always through the `Order`.

Repository: Provides an abstraction for persisting and retrieving aggregates. The domain doesn't care about database implementation.

Domain Service: Business logic that doesn't naturally belong to an entity. `TransferService.transfer(fromAccount, toAccount, amount)` — the transfer operation doesn't belong to either account.

Domain Event: Something significant that happened in the domain. `OrderPlaced`, `PaymentFailed`. Published after the aggregate state changes.

Bounded Context: A logical boundary where a domain model is defined and applicable. The `User` in the `Orders` bounded context has different attributes than `User` in the `Support` bounded context.

11.3 Clean Architecture

Clean Architecture (Robert C. Martin) organizes code into concentric layers where dependencies point inward — outer layers depend on inner layers, never the reverse.

Ports and Adapters (Hexagonal Architecture): The application core defines **ports** (interfaces). **Adapters** implement those ports for specific technologies (HTTP controller adapts to use cases, MySQL repository adapts to data port). The application core is technology-agnostic.

11.4 Code Smells & Refactoring

Code smells are surface indicators of deeper design problems. Key smells:

Code Smell	Description	Refactoring
Long Method	Method > 20 lines	Extract Method
God Class	Class knows too much, does too much	Extract Class, Single Responsibility
Feature Envy	Method uses another class's data more than its own	Move Method
Shotgun Surgery	One change requires many small changes in many classes	Move Method/Field, consolidate
Primitive Obsession	Using primitives instead of small objects	Replace Primitive with Object
Switch Statements	Complex conditionals on type codes	Replace Conditional with Polymorphism
Duplicate Code	Same code in multiple places	Extract Method, Pull Up

---

11.5 Anti-Patterns

God Object: A single class that knows too much or does too much. Violates SRP. Common in legacy code.

Singleton Abuse: Using Singleton for everything that needs global access. Creates hidden dependencies, makes testing difficult, causes concurrency issues.

Anemic Domain Model: Domain objects (entities) have only getters/setters with no business logic. All logic lives in Service classes. Looks OO but is actually procedural.

Spaghetti Code: Tangled, unstructured code with many GOTOs or deeply nested conditionals; no clear separation of concerns.

Golden Hammer: Using a familiar technology for every problem regardless of fit. "If all you have is a hammer, everything looks like a nail." (Using relational DB for everything, or Redis for persistent data.)

Chapter 11 — Quick Revision Box

> - **Producer-Consumer:** Use `BlockingQueue.put()` (blocks when full) and `take()` (blocks when empty) for backpressure

- **ReadWriteLock:** Multiple concurrent readers, exclusive writer; use when reads >> writes
- **DDD:** Entity (identity), Value Object (attributes, immutable), Aggregate (unit with root), Repository (persistence abstraction)
- **Bounded Context:** separate domain models for separate subdomains; avoid one universal model
- **Clean Architecture:** dependencies point inward; entities → use cases → adapters → frameworks
- **Common smells:** God Class (too much), Feature Envy (wrong home), Shotgun Surgery (scattered logic)
- **Anemic Domain Model:** objects with only getters/setters; business logic scattered in services (anti-pattern)

Chapter 12: Top 40 Most-Asked System Design Interview Questions {#chapter-12}

Q1: Design a URL Shortener

Question: Design a URL shortening service like TinyURL or bit.ly.

Answer: The core challenge is generating unique, collision-free 7-character codes for billions of URLs. The simplest approach is Base62 encoding of a monotonically increasing ID (from a database auto-increment or a distributed Snowflake ID generator). This eliminates collision concerns entirely. For the short code, 62^7 ≈ 3.5 trillion combinations — sufficient for decades.

The read path is the most critical: 100 million redirects/day at P99 < 10ms. Serve redirects from Redis with an LRU cache. CDN caching of popular short codes (with HTTP 302 to preserve analytics) is the key performance optimization. Use 302 (Temporary Redirect) rather than 301 (Permanent) so browsers re-query your servers on each visit, enabling click counting.

For storage, MySQL with a `url_mappings(short_code PK, original_url, user_id, expires_at, created_at)` table is sufficient. Index `MD5(original_url)` for deduplication. Shard by `short_code` prefix for scale. Analytics (clicks, referrer, geo) should be decoupled — publish click events to Kafka and aggregate asynchronously in a data warehouse.

Trade-offs: Pre-generated codes (safe against collision but complex) vs. hash-based (simple but needs collision handling). Synchronous analytics (accurate but slow) vs. asynchronous (fast but eventually consistent).

> **Quick Recall:** Base62(auto-increment ID) → unique short code; Redis for redirects; Kafka for analytics; 302 redirect for tracking.

Q2: Design a Web Crawler

Question: Design a distributed web crawler that can crawl 1 billion pages in 30 days.

Answer: A web crawler has five core components: URL Frontier, Fetcher, Parser, Deduplication, and Storage. The URL Frontier is a priority queue of URLs to crawl, organized by domain for politeness (one request per domain per second, respecting robots.txt and crawl-delay directives). The priority ordering is based on estimated page quality (PageRank approximation, recency, domain authority).

Deduplication is the biggest scalability challenge. A HashSet of seen URLs would require hundreds of GB. A Bloom filter can represent 10 billion URLs in ~12 GB with a manageable false positive rate (~0.1%). False positives mean occasionally skipping a valid URL — acceptable for a crawler. Content deduplication (detecting when two different URLs have identical content) uses MinHash or SimHash fingerprinting.

For scale, run hundreds of fetcher workers across multiple machines. DNS resolution is a bottleneck — cache DNS results aggressively. Store raw crawled HTML in object storage (S3) with a key of `domain/path/timestamp`. Extracted links are published to Kafka for the frontier workers.

Trade-offs: Broad crawl (many domains, shallow) vs. focused crawl (deep within target domains). Aggressive crawling vs. politeness (being banned from hosts). Freshness (recrawl frequently) vs. storage cost.

> **Quick Recall:** URL Frontier (priority queue) + Bloom filter (dedup) + Distributed Fetchers + robots.txt politeness.

Q3: Design a Rate Limiter

Question: Design a rate limiter for an API gateway that supports 1 million users.

Answer: A rate limiter must be: (1) accurate, (2) low latency (adds <1ms overhead), (3) distributed (works across multiple API server instances), and (4) fault-tolerant (doesn't block traffic if the limiter itself fails). The sliding window counter algorithm balances accuracy and efficiency: maintain two counters per client (current window and previous window) and interpolate.

The rate limiter should return HTTP 429 (Too Many Requests) with `Retry-After` and `X-RateLimit-Remaining` headers. For multi-tier rate limiting, maintain separate counters per: global (protect the entire system), per-user, and per-endpoint (expensive operations have lower limits).

Trade-offs: Accuracy (sliding window log) vs. memory efficiency (fixed window counter). Redis centralization (accurate, potential SPOF) vs. local counters (less accurate but no SPOF, no network overhead).

> **Quick Recall:** Sliding window counter in Redis; Lua script for atomic check+increment; 429 + Retry-After headers; per-user + per-endpoint limits.

Q4: Design a Key-Value Store

Question: Design a distributed key-value store like DynamoDB or Cassandra.

Answer: The architecture is based on consistent hashing for data distribution and replication for fault tolerance. Keys are mapped to nodes via a hash ring. Each key is replicated to N nodes (N=3) for durability. The system is AP (available + partition-tolerant) — during network partitions, it returns possibly stale data rather than returning an error.

Reads and writes use a quorum system (R + W > N for consistency). Write path: client sends PUT(key, value) to any coordinator node → coordinator identifies N responsible nodes via consistent hashing → writes to W nodes → returns success when W acknowledgments received. Read path: coordinator queries R nodes → returns value with the latest vector clock (for conflict detection).

Storage engine uses an **LSM Tree** (Log-Structured Merge Tree): writes go to an in-memory memtable → periodically flushed to immutable SSTables on disk → background compaction merges SSTables. This makes writes append-only (extremely fast I/O) at the cost of slightly slower reads (may need to check multiple SSTables).

Failure handling: Hinted Handoff (coordinator stores hints for unavailable replicas), Read Repair (coordinator repairs stale replicas during reads), anti-entropy with Merkle Trees (background process to detect and fix inconsistencies between replicas).

> **Quick Recall:** Consistent hashing + virtual nodes; N=3 replicas; quorum R+W>N; LSM Tree storage; hinted handoff + read repair for fault tolerance.

Q5: Design Twitter

Question: Design Twitter's core functionality: posting tweets, following users, and viewing timelines.

Answer: Twitter is a classic example where the write-to-read ratio is extremely imbalanced (~1:10,000). The most critical design decision is timeline generation. Fan-out-on-write (pre-computing each follower's timeline when a tweet is posted) gives fast timeline reads at the cost of expensive writes. Fan-out-on-read (computing the timeline at read time by merging followed users' tweets) gives cheap writes but expensive reads.

Twitter uses a hybrid approach: for regular users (< 1M followers), fan-out-on-write pre-populates a Redis sorted set (timeline cache) for every follower. For celebrities (> 1M followers), fan-out-on-write is cost-prohibitive — 100M writes per tweet. Instead, celebrity tweets are stored separately and merged at read time into the pre-computed timeline.

The database design shards the `tweets` table by `user_id`. A separate `follows` table records the follower graph. Timeline entries in Redis are sorted by tweet Snowflake ID (time-sortable), enabling efficient pagination.

For media (images, videos), upload directly to S3 via pre-signed URLs and serve through CDN. Tweet text and metadata go to MySQL/Cassandra.

> **Quick Recall:** Hybrid fan-out — write for regular users, read-merge for celebrities; Redis timeline cache (sorted set by tweet_id); Snowflake IDs for time-sorting.

Q6: Design WhatsApp / Facebook Messenger

Question: Design a real-time messaging system like WhatsApp.

Answer: Real-time bidirectional communication requires WebSockets — persistent TCP connections where the server can push messages to the client without polling. Each chat server maintains thousands of WebSocket connections. When Alice sends a message to Bob, Alice's WebSocket sends the message to the chat server → server checks if Bob is connected to the same server (route directly) or a different server (route via message queue) → deliver to Bob.

Message persistence uses Cassandra with `(conversation_id, timestamp)` as the partition and clustering key. This enables efficient "fetch last 50 messages for conversation X" queries. Messages are never deleted — only archived by retention policy.

Online presence detection uses heartbeats: clients send a ping every 5 seconds. The Presence Service maintains a Redis key `presence:{user_id}` with 10-second TTL. Any

service can query presence in O(1). Group chat delivery: the server fans out the message to all group members' WebSocket connections.

Message delivery states (sent ✓ / delivered ✓ ✓ / read ✓ ✓ blue) use asynchronous ACKs. The server stores the message state and updates when the recipient's device acknowledges receipt.

> **Quick Recall:** WebSocket for real-time; Cassandra for message history; Redis TTL for presence; two-tick delivery receipts via async ACK.

Q7: Design YouTube

Question: Design a video streaming platform like YouTube.

Answer: YouTube has two distinct workloads: video upload/processing (write-heavy, async) and video streaming (read-heavy, requires global CDN). The upload pipeline works as follows: user uploads raw video → stored in S3 → Kafka event triggers transcoding workers → FFmpeg transcodes to multiple resolutions (4K, 1080p, 720p, 480p, 360p) and formats (H.264, H.265/HEVC, VP9) → transcoded files stored back in S3 → video metadata updated in database → thumbnails generated.

Streaming uses HLS (HTTP Live Streaming): each video is split into 5-10 second segments. A manifest file (.m3u8) lists all available resolutions and segment URLs. The client player monitors bandwidth and dynamically requests the highest quality it can sustain without buffering. Segments are served from CDN edge nodes — the player fetches from the CDN, not YouTube's origin servers.

The database stores: video metadata (MySQL), user data (MySQL), view counts (eventually consistent counter — use Redis with periodic flush to DB), comments (MySQL with pagination), and search index (Elasticsearch with video title, description, tags).

> **Quick Recall:** Upload → S3 → Kafka → FFmpeg transcoding → multiple resolutions; HLS adaptive bitrate streaming; CDN for delivery; Elasticsearch for search.

Q8: Design Uber

Question: Design a real-time ride-sharing service like Uber.

Answer: Uber's core technical challenges are real-time location tracking, proximity search, and matching. Drivers send GPS coordinates every 4-5 seconds via WebSocket/HTTP. The Location Service stores these in Redis using geospatial commands (`GEOADD drivers:available longitude latitude driver_id`). To find drivers

near a rider: `GEORADIUS drivers:available lat lng 5 km ASC COUNT 20` returns the 20 nearest available drivers.

The matching algorithm ranks nearby drivers by: ETA (computed via OSRM routing engine), car type, driver rating, and acceptance rate. Offers are sent to drivers one at a time (or in batches) — first to the nearest driver. If the driver doesn't respond in 10 seconds, offer to the next.

Surge pricing uses demand/supply ratio per geohash cell. Aggregate ride requests and driver availability per geohash and compute a multiplier. Geohashing divides the map into cells — nearby cells share a common prefix, enabling efficient boundary queries.

Trip data (pick-up, drop-off, route, fare) is stored in PostgreSQL sharded by trip_id. Historical location data is written to a data warehouse (BigQuery) for analytics and route optimization ML training.

> **Quick Recall:** Redis GEORADIUS for nearby drivers; WebSocket for real-time location; OSRM for ETA; geohash for surge pricing; PostgreSQL (sharded) for trip history.

Q9: Design a Distributed Message Queue (Kafka-style)

Question: Design a distributed message queue that can handle millions of messages per second.

Answer: A high-throughput message queue is built around the concept of a persistent, ordered log. Messages are written to a distributed append-only log partitioned across multiple brokers. Each topic has N partitions; each partition is assigned to one broker as leader and replicated to M-1 other brokers as followers.

Producers write to the partition leader. The leader writes to its local log and replicates to followers. Once W followers acknowledge (configurable), the producer receives an acknowledgment. Consumers subscribe to partitions and track their position via offsets stored in a special internal topic or ZooKeeper/KRaft. Consumer groups allow horizontal consumer scaling — each partition is consumed by exactly one consumer per group.

High throughput is achieved via batching (producers buffer messages and send in batches), sequential disk I/O (append-only logs are sequential, an order of magnitude faster than random access), and zero-copy transfers (kernel-level sendfile to skip user-space copying).

Retention is configurable: time-based (delete after 7 days) or size-based (delete oldest when partition reaches 50 GB). Log compaction retains the latest value per key forever — used for change data capture (CDC) patterns.

> **Quick Recall:** Partition = ordered log; leader-follower replication; offset-based consumption; consumer groups for parallel consumption; batching + sequential I/O for throughput.

Q10: Design a File Storage System (like Dropbox)

Question: Design a file sync and storage service like Dropbox.

Answer: Dropbox's core innovation is efficient sync — detecting and uploading only changed bytes, not entire files. Files are split into 4 MB chunks; each chunk is identified by its SHA-256 hash. When a file changes, only the modified chunks are uploaded. The client maintains a local metadata DB (SQLite) tracking chunk hashes. On sync, it compares local chunk hashes with the server's version.

Upload flow: client splits file → computes chunk hashes → checks which chunks are already on server (dedup) → uploads only missing chunks to S3 → server records the file metadata and chunk list. Deduplication is chunk-level: if two users upload the same file, only one copy is stored (content-addressed storage).

Real-time sync uses WebSockets or long-polling. When a file changes, the server pushes a notification to all connected clients. The client fetches the changed metadata, determines which chunks to download, and downloads only those chunks.

For large files and resumable uploads, use S3 multipart upload — upload is split into concurrent parts, and failed parts can be individually retried. Metadata (file names, paths, versions, sharing permissions) is stored in MySQL.

> **Quick Recall:** Content-based chunking → SHA-256 dedup; upload only changed chunks; S3 for chunks; WebSocket push for sync notification; SQLite local metadata DB.

Q11: Design a Collaborative Document Editor (Google Docs)

Question: Design a real-time collaborative document editor like Google Docs.

Answer: The core challenge is **conflict resolution** — multiple users typing simultaneously. The established algorithm is **Operational Transformation (OT)** or, more modernly, **CRDTs (Conflict-free Replicated Data Types)**. OT transforms conflicting edits so they can be applied in any order and converge to the same result.

Example: Document "Hello World". User A deletes "Hello" at position 0. User B inserts "!" at position 5. If operations arrive out of order, OT adjusts position 5 in B's operation (since A deleted 5 chars, B's insertion position becomes 0). The document correctly shows "! World" after both operations.

The collaboration server maintains the authoritative document state. Each client connects via WebSocket and sends operations as the user types. The server applies operations using OT, broadcasts to all other clients, and persists to database. Document versions allow point-in-time recovery.

For persistence: document snapshots are stored periodically in object storage; operations are logged in a database for version history. Document content for editing is kept in-memory on the collaboration server (Redis or application memory); for large documents, partition by section.

> **Quick Recall:** OT (Operational Transformation) or CRDT for conflict-free merging; WebSocket for real-time edits; server applies OT and broadcasts to all clients; periodic snapshots + operation log for history.

Q12: Design a Search Engine

Question: Design a web search engine like Google at a high level.

Answer: A search engine has three main phases: crawling (discovering pages), indexing (processing and storing content), and querying (serving results). The crawler (discussed in Q2) continuously discovers and downloads web pages. The indexer processes raw HTML: extracts text, tokenizes and normalizes (lowercase, stemming), removes stop words, and builds an inverted index.

The inverted index maps each term to a posting list: `{term → [(doc_id, position, tf), ...]}` where `tf` is term frequency. This is stored in a distributed manner — the index is sharded by term hash across many servers. For a query "red shoes", the search service fetches posting lists for "red" and "shoes" from potentially different index shards, intersects the lists, and ranks results.

Ranking uses signals: TF-IDF (term frequency-inverse document frequency, measures how relevant a term is to a document), PageRank (how many high-quality pages link to this page), and hundreds of machine-learned features (click-through rate, page load time, content freshness, user intent signals).

Scaling: The index is replicated — multiple copies of each index shard. Query routing sends the query to all shard replicas in parallel, collects partial results, and merges (distributed query execution). Caches store results for popular queries.

> **Quick Recall:** Crawl → Parse → Inverted Index (term → posting list); sharded index; TF-IDF + PageRank ranking; parallel query across index shards.

Q13: Design a Notification System

Question: Design a system to send millions of push/email/SMS notifications daily.

Answer: The notification system must be reliable (no missed notifications), efficient (batch where possible), and respectful (user preferences, quiet hours, frequency caps). The architecture uses a pipeline: trigger source → notification service → channel-specific workers → delivery providers.

Any service can trigger a notification by publishing to a Kafka topic with the notification payload and intended audience. The Notification Service consumes from Kafka, enriches the notification (fetching user contact info, device tokens, preferences from cache/DB), checks frequency caps (using Redis counters), applies quiet hours rules, and routes to the appropriate channel queue (separate Kafka topics for push, SMS, email).

Channel workers (push, SMS, email) consume from their queue and call the respective provider API (APNs/FCM for push, Twilio for SMS, SendGrid for email). Failed deliveries are retried with exponential backoff; after max retries, the message goes to a Dead Letter Queue for investigation.

Delivery tracking: each notification gets a UUID. Workers update the delivery status (sent/delivered/failed) in a Postgres table via async events from the provider (APNs delivery receipts, SendGrid webhooks).

> **Quick Recall:** Kafka for async decoupling; per-channel workers (push/SMS/email); Redis frequency caps; exponential backoff retries; DLQ for failures; webhook-based delivery receipts.

Q14: Design a Recommendation System

Question: Design a recommendation system for an e-commerce platform.

Answer: Recommendation systems use two main approaches: collaborative filtering (recommend based on behavior of similar users) and content-based filtering (recommend items similar to what the user has liked). Most production systems use a **two-stage architecture**: candidate generation (retrieve thousands of candidates quickly) followed by ranking (score and sort using a heavy ML model).

Candidate generation: Approximate nearest neighbor search (ANN) over user/item embeddings. User and item embeddings are pre-computed by training a matrix factorization model (ALS) or a neural collaborative filtering model on user-item interaction data. At serving time, retrieve the top-K most similar items to the user's embedding using FAISS (Facebook AI Similarity Search) or a vector database.

Ranking: Pass candidates through a feature-rich model (gradient boosting or deep neural network) that considers: item features (category, price, rating), user features (age, location, purchase history), context (time of day, device), and interaction features (viewed but not bought).

Real-time personalization: Log user clickstream events to Kafka → stream processor (Flink) updates user feature vectors in real-time → serving layer combines real-time features with pre-computed embeddings.

> **Quick Recall:** Two-stage: candidate generation (ANN on embeddings) + ranking (ML scoring); ALS/NCF for embeddings; FAISS/vector DB for ANN; Flink for real-time feature updates.

Q15: Design a Job Scheduler

Question: Design a distributed job scheduler (like cron at scale, or AWS Batch).

Answer: A job scheduler allows users to submit jobs (code + config) that run at specified times or on a recurring schedule. The system must handle: job persistence (survive restarts), distributed execution (scale across many worker nodes), failure handling (retry failed jobs), and priority scheduling.

The scheduler stores jobs in a database with their schedule (cron expression or interval), next_run_time, priority, and status. A **scheduler daemon** runs a loop: query for jobs where `next_run_time <= now AND status = 'scheduled'`, acquire a distributed lock (Redis Redlock or database row lock) to prevent duplicate execution, enqueue the job to a work queue (Kafka topic by priority), and update next_run_time.

Worker nodes pull from the work queue, execute the job, and report status back. Job isolation: each job runs in a Docker container (Kubernetes Job) to prevent resource contention. Time-critical jobs (second-granularity) use a separate path from heavy batch jobs.

Failure handling: workers send heartbeats during execution; if a worker dies mid-execution (heartbeat stops), a watchdog detects this and re-enqueues the job. Idempotent jobs are safe to re-run; non-idempotent jobs need at-least-once semantics with deduplication.

> **Quick Recall:** Database stores schedule + next_run_time; distributed lock prevents duplicate execution; Kafka queue for workers; Docker/K8s for isolation; heartbeat watchdog for failure detection.

Q16: Design a Hotel Booking System

Question: Design a hotel booking system like Booking.com.

Answer: The critical challenge in hotel booking is **inventory management and double-booking prevention**. Two users should not be able to book the same room for the same night. The naive solution — read inventory then write booking — has a race condition. The correct solution uses database-level locking or optimistic concurrency control.

Inventory model: `hotel_inventory(hotel_id, room_type, date, total_rooms, reserved_rooms)`. Booking atomically increments `reserved_rooms` with a condition: `UPDATE hotel_inventory SET reserved_rooms = reserved_rooms + 1 WHERE hotel_id = ? AND date = ? AND (total_rooms - reserved_rooms) > 0`. If zero rows affected, the room is sold out.

For search, the inventory database is not queried directly. An Elasticsearch index is maintained for hotel search (location, amenities, price, availability). When inventory changes, update Elasticsearch asynchronously. The booking step verifies real-time availability in the source-of-truth DB.

Distributed transactions: booking involves multiple steps (reserve inventory, charge payment, send confirmation). Use the Saga pattern. If payment fails, trigger a compensating transaction to release the inventory reservation.

> **Quick Recall:** Atomic SQL UPDATE with conditional WHERE for inventory; Elasticsearch for search; Saga pattern for multi-step booking (inventory + payment); pessimistic or optimistic locking.

Q17: Design a Food Delivery App (Swiggy/Zomato)

Question: Design a food delivery application.

Answer: Food delivery connects three parties in real-time: customers, restaurants, and delivery partners. Key components: Order Management, Restaurant Matching, Delivery Assignment, and Real-Time Tracking.

Order flow: Customer places order → restaurant receives notification (WebSocket push or polling) and accepts/rejects → order dispatched to delivery queue → nearest available delivery partner assigned → delivery partner picks up and delivers → status updates pushed to customer at each step via WebSocket/push notification.

Delivery partner assignment: Similar to Uber's matching. Use Redis GEORADIUS to find delivery partners near the restaurant. Rank by distance and estimated pickup time. Assignment is offered to one partner at a time with a 30-second timeout.

Menu and catalog: Stored in PostgreSQL (menu items, prices, nutrition info) with Elasticsearch for search. Menu prices and availability can change rapidly — cache with a 5-minute TTL and push-invalidate on changes.

Peak load handling: Surge in orders during meal times → pre-scale delivery servers based on historical patterns (predictive autoscaling). Use a work queue (Kafka) to absorb order spikes and process them smoothly even if the restaurant and delivery systems are slower.

> **Quick Recall:** WebSocket for real-time status updates to customers; Redis GEORADIUS for delivery partner matching; Kafka for order queue; predictive autoscaling for meal-time peaks.

Q18: Design a Multiplayer Game Backend

Question: Design the backend infrastructure for a real-time multiplayer game.

Answer: Multiplayer games require ultra-low latency (< 50ms) and high consistency within a game session. The architecture uses a dedicated **Game Server** per match, running custom UDP-based communication (TCP's retransmission adds too much latency for games). WebRTC's data channel or custom UDP with application-level reliability provides the transport layer.

Matchmaking: Players enter a matchmaking queue based on skill rating (ELO/MMR). A matchmaking service groups players with similar ratings and minimal geographic latency. Once a group is formed, spin up a dedicated game server instance (or assign to a server in a pool) in the closest datacenter.

Game state management: The game server maintains authoritative game state. All player inputs (move, shoot, etc.) go to the server; the server computes the outcome and broadcasts updated state to all players. Client-side prediction (the client immediately applies the player's own input locally while awaiting server confirmation) reduces perceived latency.

Persistence: Game results (scores, kills, achievements) are persisted to the database after the match ends. Player stats use a Redis leaderboard (sorted set) updated at match end. Replay storage: record all game inputs (not state) — replays are deterministically reconstructed by replaying inputs.

> **Quick Recall:** Dedicated game server per match; UDP (not TCP) for low latency; client-side prediction; authoritative server-side state; Redis sorted set for leaderboards; record inputs (not state) for replays.

Q19: Design a Metrics Aggregation System

Question: Design a system to collect, aggregate, and alert on metrics from thousands of servers.

Answer: A metrics pipeline ingests millions of data points per second, aggregates them (sum, count, average, percentiles) over time windows, stores them efficiently, and evaluates alert conditions. The architecture follows a Lambda or Kappa approach.

Collection: Agents (StatsD, Prometheus node_exporter) on every server collect metrics and push/expose them. A central collector (Prometheus scrape or Kafka producer)

ingests raw metrics. Use UDP for non-critical metrics (fire-and-forget, no back-pressure) and TCP/HTTP for critical metrics.

Aggregation: Stream processing (Kafka Streams, Flink) consumes the raw metric stream and computes tumbling/sliding window aggregations (sum over last 1 minute, average over last 5 minutes) in real-time.

Storage: Time-series databases (InfluxDB, TimescaleDB, Prometheus TSDB) are optimized for this workload. They use columnar storage with run-length encoding for time values and delta encoding for values (CPU usage changes slowly, so deltas are small). Retention tiers: full resolution for 15 days → 1-minute aggregates for 90 days → 1-hour aggregates for 2 years.

Alerting: Rules evaluate aggregated metrics against thresholds. Use a rule engine (Prometheus AlertManager) that evaluates alert conditions periodically. Alerts route to on-call systems (PagerDuty, OpsGenie) with deduplication and silencing.

> **Quick Recall:** Agent → Kafka → Stream aggregation (Flink) → TSDB; delta encoding for compression; tiered retention (full → aggregated → archived); AlertManager with deduplication.

Q20: Design a Real-Time Bidding System (AdTech)

Question: Design a real-time bidding (RTB) system where ad exchanges auction ad impressions to bidders in < 100ms.

Answer: RTB is one of the most latency-sensitive distributed systems. The entire auction — receive bid request, evaluate it against campaigns, compute a bid price, and respond — must complete in 100ms globally.

Bid request flow: User loads a web page → publisher's ad server sends a bid request (user data, ad slot dimensions, URL) to the exchange → exchange sends the bid request to N DSPs (Demand-Side Platforms) simultaneously → each DSP evaluates and returns a bid price within 80ms → exchange picks the highest bidder → winner is notified → ad is served.

DSP architecture: Incoming bid requests are received by a stateless bid server cluster. The bid server queries a local in-memory cache (not Redis — the network round-trip is too slow) containing user segments, campaign targeting criteria, and bid prices. Pre-load all necessary data into process memory (1-2 GB) at startup.

Data pipeline: User behavioral data (page views, conversions) flows into Kafka → Spark Streaming computes user segments → pushes updated segments to all bid servers' local caches asynchronously. Campaign budget management uses atomic Redis counter decrements; when a campaign budget is exhausted, the budget service pushes a campaign-disable event to all bid servers.

> **Quick Recall:** < 100ms SLA means in-process memory cache (no Redis round-trip); stateless bid servers; pre-loaded user segments; Redis atomic decrements for budget; async data pipeline for user data updates.

Q21: Design a Distributed Lock Service

Question: Design a distributed lock to coordinate access across multiple services.

Answer: A distributed lock ensures mutual exclusion across processes on different machines. The most common approach is Redis's Redlock algorithm: acquire the lock on N/2+1 independent Redis instances within a time window. If a majority acquired, the lock is held; otherwise, release all and retry.

Single-instance Redis lock: `SET lock_key unique_id NX EX 30` (SET if Not eXists, with 30-second expiry). The expiry ensures the lock is released even if the holder crashes. Use a unique value (UUID) to ensure only the lock holder can release it (use Lua script for atomic check-and-delete).

For higher correctness guarantees, use ZooKeeper or etcd. ZooKeeper's ephemeral nodes automatically disappear when the client session ends (client crash = lock released). etcd uses a lease mechanism with similar guarantees.

> **Quick Recall:** Redis: `SET key uuid NX EX ttl`; Lua script for atomic release; Redlock for multi-instance; ZooKeeper/etcd for stronger guarantees; always set TTL to prevent deadlocks.

Q22: Design a Leader Election System

Question: How would you implement leader election in a distributed system?

Answer: Leader election ensures exactly one node acts as the coordinator. ZooKeeper uses ephemeral sequential nodes: all candidates create `/election/candidate-` (sequential), and the node with the lowest sequence number is the leader. All others watch the node with the next lower number. If that node disappears, the next-in-line is elected.

etcd provides a built-in election API based on the Raft consensus algorithm. Campaigns acquire a lease; the candidate that successfully creates a key in etcd's distributed log becomes leader. Lease expiry or client failure releases leadership.

Raft consensus: nodes elect a leader through a vote. A candidate sends a RequestVote RPC to all nodes; if it receives a majority of votes, it becomes leader. The leader sends periodic heartbeats (AppendEntries RPCs). If followers don't hear from the leader within an election timeout (150-300ms), they start a new election.

> **Quick Recall:** ZooKeeper: ephemeral sequential nodes; lowest node = leader, others watch predecessor; etcd: built-in Election API with Raft; Raft: RequestVote → majority → leader; heartbeat to maintain leadership.

Q23: Design a Logging Pipeline

Question: Design a logging pipeline for a microservices system generating 1 TB of logs/day.

Answer: A logging pipeline must handle high write throughput, enable fast search, and manage retention cost. The reference architecture is the ELK/EFK Stack.

Collection: Applications write structured JSON logs to stdout. Container runtime (Docker) captures stdout → Filebeat/Fluentd agent (deployed as DaemonSet in Kubernetes) reads container log files and ships to Kafka. Using Kafka as a buffer between log shippers and indexers is critical — without it, indexer slowdown causes backpressure that could block application logging.

Indexing: Logstash (or Flink) consumers parse raw logs, enrich them (add host, service, datacenter metadata), filter sensitive data (PII masking), and index into Elasticsearch. Elasticsearch uses time-based indices (`logs-2024.01.15`) to enable efficient retention management and search scoping.

Storage tiering: Hot tier (Elasticsearch SSD): logs from last 7 days, fully indexed. Warm tier (Elasticsearch HDD): 7-30 days, indexed but less frequently accessed. Cold tier (S3 + Athena): 30+ days, compressed and stored in S3 for ad-hoc SQL queries. Deletion policy: delete warm indices after 90 days.

> **Quick Recall:** App → stdout → Filebeat → Kafka → Logstash → Elasticsearch; Kafka buffer prevents backpressure; time-based indices for retention; S3 + Athena for cost-effective long-term storage.

Q24: Design a Music Streaming Service (Spotify)

Question: Design a music streaming service like Spotify.

Answer: Music streaming combines catalog management, licensing-aware content delivery, and personalized discovery. The catalog stores millions of tracks as audio files (typically 320 kbps Ogg Vorbis or AAC) in S3, served via CDN.

Streaming: Unlike video, audio tracks are relatively small (3-5 MB for a 3-minute song at 320 kbps). The client can buffer the entire file rather than needing adaptive bitrate streaming. However, Spotify still offers multiple quality levels (96/160/320 kbps) and adaptively streams based on connection. Track files are served directly from CDN with pre-signed URLs that expire after the session ends (DRM light).

Personalization (Discover Weekly): Spotify's recommendation engine combines collaborative filtering (users with similar listening history), content-based analysis (audio fingerprinting and BPM/genre features), and NLP on track metadata. The model runs weekly batch jobs to compute recommendations. Real-time recommendations use a lighter model with the user's recent listening session.

Offline mode: Tracks are downloaded and encrypted client-side (symmetric key derived from user credentials + device ID). The encryption key is never stored independently — it requires a valid Spotify session to derive.

> **Quick Recall:** S3 + CDN for audio files; pre-signed URLs for access control; collaborative filtering + audio features for recommendations; Kafka for playback events; offline encryption via session-derived keys.

Q25: Design a Code Deployment System

Question: Design a system to deploy code to thousands of servers reliably.

Answer: A code deployment system must: build artifacts, store them, deploy to servers without downtime, support rollback, and handle partial failures gracefully. The pipeline is: code commit → CI build → artifact registry → deployment orchestrator → rolling deployment.

Build and artifact management: CI servers (Jenkins, GitLab CI, GitHub Actions) build Docker images and push them to a registry (ECR, Artifactory). Artifacts are versioned by git SHA. Immutable artifacts — once built, the artifact never changes.

Deployment strategies: In Kubernetes, rolling updates replace old pods gradually (25% at a time by default). Deployment health is checked between batches — if error rate rises, the deployment is automatically paused. Blue-green deployments maintain two environments; a DNS or load balancer switch completes the cutover in seconds.

Safe deployment checklist: canary deploy to 1% → monitor metrics for 15 min → 10% → 50% → 100%. Feature flags decouple deployment from feature release. Database schema changes must be backward-compatible with both old and new code (never delete a column in the same deploy that removes usage of it).

> **Quick Recall:** Immutable artifacts (Docker image + git SHA); rolling update with health checks; canary → progressive rollout; blue-green for instant rollback; backward-compatible DB migrations.

Additional Interview Questions (Q26–Q40)

The following questions follow the same structured format. Due to space, key

architecture decisions are provided:

Q26: Design a Distributed ID Generator Snowflake: 41-bit timestamp + 10-bit machine + 12-bit sequence = 64-bit int. 4096 IDs/ms/machine. Clock skew requires NTP or refuse-until-clock-advances guard. Alternative: UUID v7 (RFC 9562, time-ordered, 128-bit).

> **Quick Recall:** Snowflake = timestamp+machineID+sequence; 4096 IDs/ms/machine; guard against clock skew.

Q27: Design a Pastebin Text blobs stored in object storage (S3) with content-addressed key (SHA-256 of content → deduplication). Metadata (expiry, visibility) in MySQL. Expiry via TTL field + scheduled cleanup job. High read:write ratio → CDN for public pastes.

> **Quick Recall:** S3 for content (content-addressed); MySQL for metadata; CDN for public pastes; scheduled job for expiry.

Q28: Design a Rate Limiter for API Gateway Token bucket in Redis with Lua atomic scripts. Per-user + per-IP + per-endpoint counters. Return X-RateLimit-Remaining and Retry-After headers. Fail-open on Redis failure (don't block traffic if limiter is down).

> **Quick Recall:** Token bucket in Redis; Lua for atomicity; fail-open on Redis failure; multiple limit dimensions.

Q29: Design an E-commerce Platform Product catalog (Elasticsearch for search, PostgreSQL for inventory); shopping cart (Redis — TTL-based, expires after 24h idle); checkout (Saga: reserve inventory → charge payment → create order); order fulfillment (Kafka for warehouse notification).

> **Quick Recall:** Elasticsearch for product search; Redis cart with TTL; Saga for checkout transaction; Kafka for order events.

Q30: Design a Ride-Sharing (Detailed Surge Pricing) Geohash divides map into cells. Count ride requests and available drivers per cell in last 5 minutes (Kafka + Redis counters). Surge multiplier = max(1.0, demand/supply × base_multiplier). Update surge map every 30 seconds.

> **Quick Recall:** Geohash cells; demand/supply ratio → surge multiplier; update every 30s via streaming aggregation.

Q31: Design a Real-Time Stock Trading System Order book: sorted structure per stock (bids desc, asks asc). Matching engine is single-threaded (to avoid race conditions) per stock. Orders arrive via Kafka; matched trades go to settlement. Market data broadcast via WebSocket to subscribers.

> **Quick Recall:** Single-threaded matching engine per stock; sorted order book; Kafka for order intake; WebSocket for market data.

Q32: Design a Social Network (Facebook) Graph DB (or adjacency list in MySQL) for friend graph. NewsFeed via hybrid fan-out. Posts in Cassandra. Photos in S3 + CDN. Friend recommendations via graph traversal (friends-of-friends). Privacy checks cached per user.

> **Quick Recall:** Adjacency list for friend graph; hybrid fan-out for feed; Cassandra for posts; S3+CDN for photos.

Q33: Design a Content Moderation System Async pipeline: upload → Kafka → ML model (image/text classifier) → auto-approve/reject or queue for human review. Human review tool shows borderline cases. Model confidence threshold determines routing. Audit trail of all moderation decisions.

> **Quick Recall:** Async Kafka pipeline; ML classifier gates; human review queue for borderline cases; confidence threshold routing.

Q34: Design an IoT Data Platform Devices → MQTT broker (Mosquitto, AWS IoT Core) → Kafka → Stream processor (Flink) → TSDB (InfluxDB) + Alert engine. Device registry (metadata, certificates) in MySQL. Downlink commands via MQTT retained messages.

> **Quick Recall:** MQTT for device comms; Kafka for ingestion; Flink for stream processing; InfluxDB for storage; alerts on aggregated metrics.

Q35: Design a Fraud Detection System Real-time: transaction → ML model inference (<100ms) → approve/decline. Features: velocity (how many txns in last 1hr), location change (impossible travel), device fingerprint, merchant category. Kafka Streams for real-time feature computation. Batch: daily retraining on labeled fraud data.

> **Quick Recall:** Real-time ML inference (<100ms); Kafka Streams for velocity features; impossible travel detection; daily batch model retraining.

Q36: Design a Healthcare Appointment System Appointment slots stored as resources with time-based locking. Optimistic locking with version field prevents double-booking. Patient medical records in HIPAA-compliant encrypted storage. Appointment reminders via notification pipeline 24h + 1h before.

> **Quick Recall:** Optimistic locking for slot reservation; encrypted storage for medical records; HIPAA compliance (encryption at rest + in transit + audit logs).

Q37: Design a Digital Wallet Double-entry ledger as source of truth. Transactions: debit + credit in same atomic DB transaction. Balance = sum of all transactions (materialized as balance field for performance). Transaction history in Cassandra (append-only). Daily reconciliation against bank APIs.

> **Quick Recall:** Double-entry ledger; atomic debit+credit; materialized balance for reads; reconciliation against external systems.

Q38: Design a Video Conferencing System (Zoom) WebRTC for peer-to-peer audio/video. For large meetings (>10 participants): SFU (Selective Forwarding Unit)

server receives all streams and forwards selectively. TURN server for NAT traversal. Recording: SFU records media streams to S3. Whiteboard: CRDT-based shared canvas.

> **Quick Recall:** WebRTC for transport; SFU for multi-party (forwards selected streams); TURN for NAT traversal; CRDT for shared whiteboard.

Q39: Design a Blockchain-based System (Brief) Distributed ledger where each block contains: hash of previous block, Merkle root of transactions, timestamp, nonce. Miners compete to find nonce such that SHA-256(block) < target (proof of work). Longest chain wins (consensus). Smart contracts: code executed deterministically on all nodes.

> **Quick Recall:** Linked blocks via hash; Merkle tree for transaction integrity; PoW consensus; smart contracts = deterministic code on all nodes.

Q40: Design a Global Content Delivery System Multi-region object storage (S3 cross-region replication). CDN with edge locations globally. Cache-Control headers (max-age=86400 for static assets). Origin Shield (single origin pull per region). Real-time CDN cache invalidation via API for deployments.

> **Quick Recall:** S3 cross-region replication; CDN with origin shield; aggressive browser caching; API-driven cache invalidation at deploy time.

Chapter 12 — Quick Revision Box

> - **URL Shortener:** Base62(ID) + Redis cache + Kafka analytics + 302 redirect
> - **Chat:** WebSocket + Cassandra (partition=conversation) + Redis presence
> - **YouTube:** Transcoding pipeline (FFmpeg) + HLS segments + CDN
> - **Uber:** Redis GEORADIUS + geohash surge + OSRM ETA
> - **Payment:** Idempotency key + double-entry ledger + Saga pattern + PCI compliance
> - **Distributed lock:** Redis `SET NX EX` + Lua release + ZooKeeper for stronger guarantees
> - **Rate Limiter:** Token bucket in Redis + Lua atomicity + fail-open on Redis failure

---

13.1 Study Platforms

Platform	What to Practice	Link	Why Use It
System Design Primer	HLD concepts, case studies, back-of-envelope practice	github.com/donnemartin/system-design-primer	Free, comprehensive, community-maintained; best starting point
ByteByteGo (Alex Xu)	Visual HLD explanations, newsletter, paid video course	bytebytego.com	Best visual explanations; covers all top interview topics
Designing Data-Intensive Applications	Deep distributed systems theory	Book by Martin Kleppmann	The definitive reference for understanding why, not just what
System Design Interview Vol. 1 & 2	30+ system design walkthroughs with diagrams	Alex Xu books	Most interview-aligned book; covers exact questions asked
Pramp	Free mock system design + coding interviews with peers	pramp.com	Real interview experience with feedback; free
Exponent	Video walkthroughs, mock interviews, system design course	tryexponent.com	Structured curriculum; great for PM + SWE

Platform	What to Practice	Link	Why Use It
Interviewing.io	Anonymous mock interviews with real engineers from FAANG	interviewing.io	Real interview simulation; brutal but accurate feedback
SystemDesignSchool.io	Interactive system design learning with quizzes	systemdesignschool.io	Good for structured learning with assessments
High Scalability Blog	Real-world architecture case studies from production companies	highscalability.com	Learn how real companies (YouTube, Twitter, Instagram) actually built their systems

---

13.2 Top YouTube Videos for Last-Minute Revision

#	Video Title	Channel	Duration	Why It's Good
1	System Design Interview: Design a URL Shortener	ByteByteGo	~30 min	Best for back-of-envelope calculations and database sharding walkthrough
2	System Design for Beginners	ByteByteGo	~45 min	Comprehensive overview of all HLD fundamentals in one video
3	Designing a Distributed System	Hussein Nasser	~25 min	Great for understanding consensus, replication, and distributed concepts
4	System Design: Design Twitter	System Design Interview	~40 min	Classic fan-out problem; covers caching, database design, and timeline generation

#	Video Title	Channel	Duration	Why It's Good
5	CAP Theorem Explained	ByteByteGo	~15 min	Quick but deep explanation with real database examples
6	Load Balancing Explained	Hussein Nasser	~20 min	Covers all LB algorithms with clear visual explanations
7	Database Sharding Explained	ByteByteGo	~20 min	Best visual explanation of range, hash, and geo sharding strategies
8	Redis Explained	ByteByteGo	~25 min	Deep dive into caching patterns, Redis data structures, and cluster setup
9	Kafka Explained in 10 Minutes	Confluent (Official)	~10 min	Quick revision of Kafka partitions, consumer groups, and delivery semantics
10	Microservices vs Monolith	ByteByteGo	~20 min	Trade-offs with real-world migration examples and when to use each
11	Design a Chat Application	System Design Interview	~35 min	Covers WebSocket, message delivery receipts, presence, and group chat
12	System Design Mock Interview	interviewing.io	~60 min	Real interview simulation with live feedback from a FAANG engineer

13.3 Books

Book	Author	Why Read It
Designing Data-Intensive Applications	Martin Kleppmann	The bible of distributed systems; explains the "why" behind every design decision
System Design Interview – An Insider's Guide (Vol. 1)	Alex Xu	Covers 16 classic system design questions with detailed diagrams

Book	Author	Why Read It
System Design Interview – An Insider's Guide (Vol. 2)	Alex Xu, Sahn Lam	Advanced topics: proximity service, payment system, stock exchange, hotel booking
Clean Code	Robert C. Martin	Essential for LLD; code quality, naming, refactoring
Design Patterns: Elements of Reusable OO Software	GoF (Gang of Four)	The original 23 design patterns reference; dense but authoritative
The Art of Scalability	Martin Abbott, Michael Fisher	Scalability patterns and organizational aspects of scaling systems

Chapter 14: System Design Glossary A–Z {#chapter-14}

A

ACID: Atomicity, Consistency, Isolation, Durability — the four properties of reliable database transactions. Ensures correctness even in the face of failures, concurrent access, and partial execution.

Actor Model: Concurrency model where "actors" are the unit of computation. Actors communicate only via messages, never sharing memory. Each actor processes one message at a time. Akka implements the actor model for JVM.

A/B Testing: Running two versions of a feature (A and B) simultaneously with different user groups to measure which version performs better on a metric (click-through rate, conversion). Requires feature flags and statistical significance measurement.

Availability: The percentage of time a system is operational and able to handle requests. Expressed in "nines" (99.9% = three nines = 8.76 hours downtime/year).

B

BASE: Basically Available, Soft state, Eventually consistent — the opposite of ACID. Describes the consistency model of many NoSQL databases (Cassandra, DynamoDB). Prioritizes availability over immediate consistency.

Backpressure: A mechanism where a downstream consumer signals to an upstream producer to slow down when it cannot keep up with the incoming data rate. Prevents

out-of-memory errors in streaming pipelines. `BlockingQueue.put()` in Java implements backpressure by blocking the producer when the queue is full.

Bloom Filter: A probabilistic data structure that tests set membership with possible false positives but no false negatives. Space-efficient: 10 billion items stored in ~12 GB. Used in web crawlers (URL deduplication), databases (avoid disk reads for non-existent keys), and spam filters.

Blue-Green Deployment: A deployment strategy maintaining two identical production environments. New version is deployed to the inactive environment (Green), tested, then traffic is switched. Enables instant rollback by switching back.

Bulkhead Pattern: Isolating components into separate resource pools (thread pools, connection pools) so a failure in one pool doesn't cascade to others. Named after ship compartments that prevent flooding.

C

CAP Theorem: A fundamental theorem stating that a distributed system can guarantee at most 2 of: Consistency, Availability, Partition Tolerance. Since network partitions are unavoidable, the real choice is between C and A during a partition.

Canary Deployment: Gradual rollout of a new software version to a small subset of users (1-5%), monitoring for issues before full rollout. Minimizes blast radius of bad deployments.

CDC (Change Data Capture): Capturing and streaming database change events (inserts, updates, deletes) to downstream consumers. Used to keep caches, search indexes, and data warehouses in sync with the source database. Implemented by Debezium (for MySQL/PostgreSQL) via the database's replication log (binlog/WAL).

Circuit Breaker: A pattern that prevents cascading failures by stopping requests to a failing service for a timeout period. Three states: Closed (normal), Open (fail-fast), Half-Open (test recovery).

Consistent Hashing: A hashing technique for distributing keys across nodes such that only K/N keys need to be remapped when a node is added or removed (vs. nearly all keys in naive modular hashing). Uses a hash ring with virtual nodes.

CQRS (Command Query Responsibility Segregation): Architectural pattern that separates the models for reading data (queries) from writing data (commands). Allows independent scaling and optimization of each path.

CORS (Cross-Origin Resource Sharing): Browser security mechanism that restricts HTTP requests to scripts to same-origin by default. Servers opt-in to cross-origin requests via response headers (`Access-Control-Allow-Origin`). Preflight OPTIONS request is sent first for non-simple requests.

D

Dark Launch: Releasing a feature to production without making it visible to users. Used to test performance under real traffic. Similar to feature flags but with no user exposure.

Denormalization: Intentionally adding redundant data to a database schema to avoid expensive joins at query time. Trades write complexity and storage for read performance.

Distributed Tracing: Tracking a single request as it propagates across multiple services. Each service receives and passes a Trace ID; individual spans represent work units. Visualized as a waterfall diagram to identify latency bottlenecks.

DLQ (Dead Letter Queue): A queue where messages that fail processing after maximum retries are deposited for investigation. Prevents poison-pill messages from blocking the main queue.

Durability: The guarantee that committed transactions survive system failures. Achieved via write-ahead logging (WAL) — changes are written to an append-only log before being applied to data files.

E

Error Budget: The allowed downtime derived from an SLO. `Error budget = 100% - SLO`. If SLO is 99.9%, the monthly error budget is 43.2 minutes. When the budget is exhausted, deployments are frozen to prioritize reliability.

ETL (Extract, Transform, Load): Data pipeline pattern for data warehouses. Extract from source → Transform (clean, join, aggregate) → Load into warehouse. Transformation happens before loading.

ELT (Extract, Load, Transform): Modern variant where raw data is loaded into a data lake first, then transformed in-place using the warehouse's compute (BigQuery, Snowflake). More flexible for exploratory analytics.

Event Sourcing: Storing domain state as a sequence of events rather than current state. Current state is derived by replaying events. Provides a complete audit log and enables time-travel queries.

F

Fan-out: Distributing one message to multiple recipients. In news feed systems, fan-out-on-write pre-writes a post to all followers' timelines when the post is created.
Fan-in: Aggregating multiple input streams into one. The opposite of fan-out; used in reduce operations and data aggregation.

Feature Flag (Feature Toggle): A configuration mechanism to enable/disable features at runtime without code deployment. Enables dark launches, A/B tests, and gradual rollouts.

Federation: Horizontal database partitioning by function or domain (separate databases for users, orders, products). Each database is independently scalable. Cross-domain joins require application-side aggregation.

G

Gossip Protocol: A distributed communication protocol where nodes periodically exchange state information with a random subset of peers. Eventually, information spreads to all nodes (like gossip). Used by Cassandra, Consul, and DynamoDB for cluster membership and failure detection.

GraphQL: A query language and runtime for APIs that allows clients to request exactly the data they need. Eliminates over-fetching and under-fetching. Single endpoint vs. multiple REST endpoints. Schema-first development.

gRPC: Google's Remote Procedure Call framework using HTTP/2 and Protocol Buffers. Binary serialization (compact, fast), supports bidirectional streaming, strongly typed via .proto schema. Ideal for high-throughput internal microservice communication.

H

Heartbeat: Periodic signal sent by a node to indicate it is alive. If a node doesn't send a heartbeat within a timeout, it is considered dead. Used by load balancers (health checks), distributed coordinators (Zookeeper), and chat presence systems.

Horizontal Scaling (Scale Out): Adding more machines to handle increased load. Requires stateless application design and a load balancer. No theoretical upper limit.

Hotspot: A specific key, partition, or server that receives disproportionately more traffic than others. Causes performance bottlenecks. Mitigated by better key distribution (random suffixes, consistent hashing) or caching.

I

Idempotency: The property of an operation where applying it multiple times produces the same result as applying it once. Essential for safe retries. GET, PUT, and DELETE are idempotent; POST typically is not. Achieved via idempotency keys that deduplicate requests.

Idempotency Key: A unique client-generated identifier sent with non-idempotent requests. The server stores the result against this key and returns the stored result for duplicate requests, preventing double-execution.

K

Kappa Architecture: A data processing architecture that uses a single streaming layer (e.g., Kafka Streams, Flink) for both real-time processing and batch reprocessing (by replaying historical events). Simpler than Lambda Architecture (no separate batch layer).

Lambda Architecture: Data processing architecture with three layers: batch layer (complete, accurate view over all historical data), speed layer (real-time view with approximate recent data), and serving layer (merges results). Complex to maintain due to dual code paths.

L

Long Polling: An HTTP polling technique where the server holds the connection open until new data is available (or a timeout). More efficient than regular polling. Simpler than WebSockets but higher latency.

LSM Tree (Log-Structured Merge Tree): A data structure used in write-optimized storage engines (Cassandra, RocksDB, LevelDB). Writes go to in-memory memtable → flush to immutable SSTables → background compaction. Makes writes sequential (fast) at the cost of read amplification (may need to check multiple SSTables).

M

MTBF (Mean Time Between Failures): Average time a system runs before failing. A measure of reliability. Higher MTBF = more reliable.

MTTR (Mean Time To Recover): Average time to restore a system after a failure. A measure of operational efficiency. Lower MTTR = faster recovery.

Merkle Tree: A tree where every leaf node is the hash of a data block, and every non-leaf node is the hash of its children. Used to efficiently compare data between two replicas — if the root hash differs, recursively compare subtrees to find which block differs. Used in Cassandra and Bitcoin.

O

OLAP (Online Analytical Processing): Workload characterized by complex queries, aggregations, and historical analysis over large datasets. Column-oriented storage (reads only needed columns). Examples: BigQuery, Redshift, ClickHouse.

OLTP (Online Transaction Processing): Workload characterized by high-volume, short-duration transactions (INSERT/UPDATE/DELETE). Row-oriented storage. Examples: MySQL, PostgreSQL.

Outbox Pattern: Ensuring reliable event publishing alongside database writes. Write both the business data and the event to the same database transaction (outbox table). A

separate poller reads from the outbox and publishes to the message broker. Prevents the dual-write problem (writing to DB succeeds but event publish fails, or vice versa).

P

PACELC Theorem: Extension of CAP: during Partition (P) choose Availability (A) or Consistency (C); Even during normal operation (E), choose Latency (L) or Consistency (C). Captures the latency-consistency trade-off present even without failures.

Partition Tolerance: In a distributed system, the ability to continue functioning even when network partitions (message loss or delay between nodes) occur.

Post-mortem (Blameless): A structured retrospective after an incident. "Blameless" means focusing on systemic causes, not individual mistakes. Produces a timeline, root cause analysis, and action items to prevent recurrence. Blameless culture encourages engineers to report and learn from incidents without fear of punishment.

Pub/Sub (Publish/Subscribe): Messaging pattern where publishers send messages to topics (not directly to subscribers). Subscribers receive all messages published to their subscribed topics. Decouples publishers and subscribers.

Q

Quorum: The minimum number of nodes that must agree for an operation to proceed. In a system with N replicas, a quorum of W = N/2+1 for writes and R = N/2+1 for reads ensures no read sees stale data (R + W > N).

R

Rate Limiting: Controlling the rate at which clients can make requests. Protects services from overload and abuse. Algorithms: token bucket (allows bursting), leaky bucket (smooth output), sliding window counter (approximate, memory-efficient).

Read Replica: A copy of a primary database that handles read traffic. Replication is typically asynchronous, allowing replication lag. Dramatically increases read scalability; writes still go to primary.

Replication: Creating and maintaining copies of data on multiple servers for fault tolerance and read scalability. Synchronous (strong consistency, higher latency) vs. asynchronous (lower latency, possible data loss in crash).

RPO (Recovery Point Objective): Maximum acceptable amount of data loss measured in time. If RPO = 1 hour, the system can lose at most 1 hour of data in a disaster. Determines backup frequency.

RTO (Recovery Time Objective): Maximum acceptable time to restore service after a failure. If RTO = 4 hours, the system must be operational within 4 hours of a disaster.

Runbook: A set of documented procedures for handling specific operational scenarios (restarting a service, responding to an alert, recovering from a failed deployment). Reduces on-call incident resolution time.

S

Saga Pattern: A sequence of local transactions for distributed transaction management. If a step fails, compensating transactions undo previous steps. Two implementations: choreography (event-driven) and orchestration (central coordinator).

Server-Sent Events (SSE): A one-way HTTP streaming protocol where the server pushes updates to the client over a persistent connection. Simpler than WebSockets for unidirectional data push (stock prices, notifications, live scores).

Sharding: Horizontal database partitioning where different rows are stored on different servers. Enables write scalability beyond a single master. Shard key selection is critical for even distribution.

SLA (Service Level Agreement): Contractual commitment between provider and customer with defined uptime and performance targets. Violations result in credits or refunds. External-facing version of SLO.

SLI (Service Level Indicator): A quantitative measure of service quality: availability (% of successful requests), latency (P99 response time), error rate.

SLO (Service Level Objective): Internal target for an SLI. "99.9% of requests return in < 200ms over 30 days." Basis for error budget calculation.

SOA (Service-Oriented Architecture): Predecessor to microservices. Services communicate via an **ESB (Enterprise Service Bus)** — a centralized bus that handles routing, transformation, and orchestration. ESB became a bottleneck; microservices replaced SOA with direct service-to-service communication.

SPOF (Single Point of Failure): A component whose failure causes the entire system to fail. Eliminated through redundancy: active-passive failover, active-active clustering, or eliminating the component entirely (e.g., replacing synchronous calls with async messaging).

T

Throttling: Reducing the rate of processing to protect a downstream service. Different from rate limiting — throttling is applied by the *provider* to protect itself; rate limiting is applied to *clients* to prevent abuse.

Toil: Repetitive, manual operational work that doesn't add permanent value (restarting services, triaging the same alert repeatedly). Google's SRE practice aims to keep toil below 50% of an engineer's time.

TCC (Try-Confirm-Cancel): A distributed transaction pattern in three phases: Try (reserve resources), Confirm (commit all), Cancel (roll back if any Try failed). More explicit than 2PC.

Two-Phase Commit (2PC): A distributed transaction protocol ensuring all participants commit or all abort. Phase 1: Coordinator asks all participants "are you ready?" Phase 2: Coordinator sends commit/abort. *Downside:* Blocking — if coordinator crashes after phase 1, participants are stuck. Rarely used in modern microservices.

V

Vector Clock: A mechanism to track causality in distributed systems. Each event is timestamped with a vector of logical clocks (one per node). Comparing vectors determines if events are causally related or concurrent. Used in DynamoDB and Riak for conflict detection.

Vertical Scaling (Scale Up): Increasing the resources (CPU, RAM) of an existing machine. Simple but has a physical ceiling and introduces a SPOF.

W

WebSocket: A full-duplex, bidirectional communication protocol over a single TCP connection. The browser upgrades an HTTP connection to WebSocket with an `Upgrade: websocket` header. Enables real-time applications: chat, gaming, live dashboards, collaborative editing.

Webhook: HTTP callback — when an event occurs, a service sends an HTTP POST to a pre-configured URL. Allows systems to react to events without polling. Used by Stripe (payment events), GitHub (push events), Twilio (SMS delivery receipts).

Chapter 14 — Quick Revision Box

> - **ACID** (SQL transactions) vs **BASE** (NoSQL eventual consistency) — know when each is appropriate
> - **CAP** → P is mandatory → choose C or A; **PACELC** adds Latency vs Consistency even without partitions
> - **RPO** = max data loss (drives backup frequency); **RTO** = max recovery time (drives redundancy design)
> - **Idempotency** enables safe retries; **Idempotency key** deduplicates non-idempotent operations
> - **SPOF** = single point of failure; eliminate via redundancy, async messaging, or active-active clustering
> - **Saga** = distributed transactions via compensating transactions; **2PC** = blocking, rarely used

> - **Gossip protocol** = epidemic information spread; **Heartbeat** = liveness signal; **Vector clock** = causality tracking

---

Final Interview Checklist

```
30 minutes before your system design interview:

REQUIREMENTS
☐ Ask: Scale? Read-heavy or write-heavy? Consistency requirements? Latency targets?
☐ Separate functional (features) from non-functional (NFRs)
☐ Agree on scope before designing

ESTIMATION
☐ Calculate QPS (daily requests / 86,400)
☐ Derive peak QPS (2-3× average)
☐ Calculate storage (daily write rate × record size × 365 × replication factor)
☐ Calculate bandwidth (QPS × average payload size)

DATA MODEL
☐ Choose SQL (ACID, complex queries, stable schema) vs NoSQL (scale, flexible, simple access patterns)
☐ Define primary keys and indexes
☐ Consider sharding strategy if needed

HIGH-LEVEL DESIGN
☐ Start with: Client → CDN → Load Balancer → App Servers → Cache → Database
☐ Add message queues for async processing
☐ Add monitoring and observability

DEEP DIVE
☐ Identify the hardest problem (consistency, scale, real-time, deduplication)
☐ Propose solution with justification
☐ Discuss trade-offs explicitly

TRADE-OFFS (always mention)
☐ Consistency vs. Availability
☐ Latency vs. Throughput
☐ Read performance vs. Write performance
```

```
□ Complexity vs. Simplicity
□ Cost vs. Performance
```

This guide covers the complete spectrum of system design interview topics. Read it twice — once for concepts, once for patterns. Good luck!

Document Statistics:

- Chapters: 14
- Systems Designed (HLD): 20+
- Design Patterns Covered: 23 (complete GoF)
- Interview Q&A: 40 questions
- Glossary Terms: 80+
